Algoritmi i strukture podataka 2022/2023.

Minimalni nivo – 15 poena

Luka Šešelja

1. Induktivno-rekurzivna konstrukcija – pojam, ilustrovanje kroz primere

Konstrukcija algoritama veoma tesno je povezana sa dokazivanjem teoreme matematičkom indukcijom.

Matematicka indukcija:

$$(P(0) \wedge (\forall n)(P(n) \Rightarrow P(n+1))) \Rightarrow (\forall n)(P(n))$$

- Baza indukcije: $P(0)$
- Induktivni korak: $(\forall n)(P(n) \Rightarrow P(n+1))$

Induktivno-rekurzivni pristup podrazumeva da se rešenje problema veće dimenzije pronalazi tako što rešimo problem istog oblika, ali manje dimenzije i od rešenja tog problema dobijemo rešenje polaznog problema.

Za početne dimenzije problema rešenje moramo izračunati direktno, bez daljeg svođenja na probleme manje dimenzije (izlaz iz rekurzije).

Ako se dimenzija uvek smanjuje → garantovano zaustavljanje.

Problem se može rešiti induktivno (iterativno) ili rekurzivno.

Induktivni (iterativni) pristup: promenljivama unutar petlje se iterativno ažurira vrednost krenuvši od rešenja elementarnih problema, pa do rešenja zadatog problema.

Rekurzivni pristup: funkcija sama sebe poziva kako bi rešila kako bi rešila problem manje dimenzije (osim u slučaju elementarnih problema koji se direktno rešavaju → izlaz iz rekurzije) i koristi ta rešenja za rešavanje početnog problema.

Primer: suma niza brojeva

Rekurzivno:

```
int zbir(int a[], int n) {
      if (n == 0)
            return 0;
      else
            return zbir(a, n-1) + a[n-1];
}
```

Iterativno:

```
int zbir(int a[], int n) {
      int zbir = 0;
      for (int i = 0; i < n; i++)
            zbir += a[i];

      return zbir;
}
```

---

2. Invarijante petlje – pojam, ilustrovana kroz premiere

Invarijanta petlje je logički uslov koji važi pre petlje, nakon svakog izvršavanja naredbi u telu petlje i nakon izvršavanja cele petlje. Garantuje nam korektnost algoritma koji ta petlja implementira.

```
<inicijalizacija>
// ovde vazi <invarijanta>
while (<uslov>) {
      // ovde vazi i <uslov> i <invarijanta>
      <telo>
      // ovde vazi <invarijanta>
}
// ovde ne vazi <uslov>, vazi <invarijanta>
```

Da bismo dokazali da je neki uslov invarijanta petlje, dovoljno je dokazati:

1. da taj uslov važi pre prvog ulaska u petlju (**odgovara bazi indukcije**)
2. da iz pretpostavke da uslov važi pre nekog izvršavanja tela petlje i da je uslov petlje ispunjen dokažemo da taj uslov važi i nakon izvršavanja tela petlje. (**odgovara induktivnom koraku**)

Ova dva tvrđenja nam garantuju da će uslov važiti i nakon izvršavanja cele petlje ako se ona ikad zaustavi. Na kraju dokazujemo i trece tvrđenje:

3. Iz toga da invarijanta važi nakon završetka petlje i da uslov petlje nike ispunjen sledi korektnost algoritma.

Primer: određivanje minimalnog elements niza

Invarijanta: nakon *i* izvršavanja tela petlje, promenljiva m ima vrednost minimuma prvih *i + 1* elemenata niza.

```
int minNiza (int a[], int n) {
      int m = a[0];
      for (int i = 1; i < n; i++)
            m = min(m, a[i]);
      return m;
}
```

- baza: nakon 0 izvršavanja petlje *m* ima vrednost minimumajednog elementa niza - važi jer smo inicijalizovali *m* na prvi element niza.
- korak: invarijanta važi pre tela petlje i važi *i* < *n*, onda će važiti i nakon izvršavanja tela petlje - važi jer znamo da pre izvršavanja *m* ima vrednost minimuma prvih *i* elemenata niza zatim ga uporedimo sa *i* plus prvim elementom niza pa će nakon izvršavanja m biti minimum prvih *i + 1* elemenata niza.
- kad se petlja završi *i* će imati vrednost *n* pa na osnovu varijante *m* ima vrednost minimuma prvih *n* elemenata niza.

---

3. Složenost – vrste složenosti, asimptotska analiza

Svaki program tokom izvršavanja zahteva određene resurse:

- procesorsko vreme
- memoriju
- energiju (bateriju)
- …

Složenost programa (ili dela programa) označava količinu resursa potrebnih da se program (ili deo programa) izvrši. Najčešće su to vreme koje je potrebno za izvršavanje i memorija koja se zauzima (ali može biti još resursa, npr. kod mobilnih uređaja važan resurs je utrošena energija). Dakle, obično se razmatraju **vremenska** i **prostorna** (**memorijska**) složenost.

Potrebni resursi zavise od dimenzije ulaza koji se obrađuju (obično obrada većih ulaza zahteva više resursa). Za različite ulaze iste dimenzije program može zahtevati različitu količinu resursa (pre svega vremena), zato se obično analizira:

- **Složenost najgoreg slučaja** – najveća kolišina resursa potrebna da se izvrši program za ulaz date dimenzije. Ovaj pristup može biti previše pesimističan, ali nam sa drudge strane garantuje da ako je program dovoljno efikasan u najgorem slučaju, biće dovoljno efikasan i za sve ostale slučajeve
- **Prosečna složenost** – količina resursa koja je u proseku potrebna da se izvrši programa date dimenzije. Da bi ova analiza bila efikasna potrebno je dobro poznavati prostor dopuštenih ulaznih vrednosti ali i verovatnoću da se svaka od tih vrednosti pojavi na ulazu.
- **Složenost najboljeg slučaja** – previše optimistična i nikada nema smisla
- **Amortizovana analiza složenosti** – nekada se tokom programa izvršava ista operacija više puta i svaki put kada se izvrši na ulazu iste dimenzije može da zahteva različitu količinu resursa. Ovom analizom složenosti određujemo potrebnu količinu resursa da se *n* takvih operacija izvrši jedna za drugom. Nije nam bitna količina resursa za izvršavanje pojedinačne operacije, već samo zbirna količina za izvršavanje svih operacija. Amortizovanu količinu resursa za jednu operaciju možemo proceniti tako što podelimo ukupnu količinu sa *n*.

Potrebna količina resursa se može egzaktno izmeriti puštanjem programa i merenjem, ali je poželjno proceniti potrebnu količinu resursa i bez pokretanja programa. Za ovo koristimo **asimptotsku analizu**. Potrebno je konstruisati **funkciju koja opisuje zavisnost potrebne količine resursa od date veličine ulaza** (obično u najgorem slučaju), zanemarujući konstantne faktore (npr. da li je jedan program uvek dva puta sporiji od drugog nije bitno u ovakvim procenama, samo je bitno da li količina resursa kod oba programa linearno zavisi od dimenzije ulaza) i oceniti njeno **asimptotsko ponašanje**.

Gornja granica složenosti se izražava korišćenjem **O-notacije:**

- funkcija f je O(g) ako "f ne raste asimptotski brže od g" tj. Ako postoji konstanta *C* tako da je za svako *n* počevši od nekog n_0 važi da je $f(n) \leq Cg(n)$

Pored O-notacije koriste se i **Ω-notacija** koja daje donju granicu ("f ne raste asimtotski sporije od g") i **Θ-notacija** koja daje precizan opis asimptotskog ponašanja ("f raste asimptotski istom brzinom kao g").

---

4. Matematičke osnove izračunavanja složenosti – osnovne formule

Aritmetički niz

- **Gausova formula**

$$1 + 2 + \ . \ . \ . \ + n = \sum_{k=0}^{n} k = \frac{n(n+1)}{2}$$

- **K-ti član aritmetičkog niza**

$$a_k = a_0 + k \cdot d$$

- **Zbir aritmetičkog niza**

$$a_0 + (a_0 + d) + (a_0 + 2d) + \ . \ . \ . \ + (a_0 + (n - 1) \cdot d) = \sum_{k=0}^{n-1} (a_0 + k \cdot d) = n \cdot a_0 + d\frac{n(n-1)}{2}$$

Geometrijski niz

- **K-ti član geometrijskog niza**

$$b_k = b_0 \cdot q^k$$

- **Zbir geometrijskog niza**

$$b_0 + b_0 \cdot q + b_0 \cdot q^2 + \ . \ . \ . \ + b_0 \cdot q^{n-1} = \sum_{k=0}^{n-1} b_0 \cdot q^k = b_0 \cdot \frac{1-q^n}{1-q}$$

- **Zbir geometrijskog reda za** $|q| < 1$

$$\sum_{k=0}^{\infty} b_0 q^k = b_0 \cdot \frac{1}{1-q}$$

- **Za** $b_0 = 1$ **i** $q = 2$ **važi**

$$1 + 2 + \ . \ . \ . \ + 2^{n-1} = 2^n - 1$$

Zato potpuno balansirano drvo sa n nivoa ima $2^n - 1$ čvorova i 2^{n-1} listova

- **Za** $b_0 = 1$ **i** $q = \frac{1}{2}$ **važi**

$$1 + \frac{1}{2} + \frac{1}{4} + \ . \ . \ . \ + (\frac{1}{2})^n = 2 - (\frac{1}{2})^n$$

Stepene sume

- **Zbir kvadrata svih prirodnih brojeva od 1 do n**

$$1^2 + 2^2 + \ . \ . \ . \ + n^2 = \sum_{k=1}^{n} k^2 = \frac{n(n+\frac{1}{2})(n+1)}{3} \approx \frac{n^3}{3}$$

- **Zbir kubova svih prirodnih brojeva od 1 do n**

$$1^3 + 2^3 + \ . \ . \ . \ + n^3 = \sum_{k=1}^{n} k^3 = (\frac{n(n+1)}{2})^2 \approx \frac{n^4}{4}$$

Kombinatorika

- **Broj načina da se iz skupa od n različitih brojeva izaberu 2 broja**

$$\binom{n}{2} = \frac{n(n-1)}{2}$$

- **Broj načina da se iz skupa od n različitih brojeva izaberu 3 broja**

$$\binom{n}{3} = \frac{n(n-1)(n-2)}{3 \cdot 2}$$

5. Klase složenosti

- **Konstantna složenost O(1)** – algoritmi čija brzina izvršavanja ne zavisi od dimenzije ulaza. Za sve ulaze se praktično izvršavaju momentalno (npr. računanje matematičke formule)
- **Logaritamska složenost O($log\ n$)** – izuzetno efikasni algoritmi koji sve moguće veličine ulaza obrade u jako malom broju koraka (npr. binarna pretraga)
- **Korenska složenost O($\sqrt{n}$)** – veoma efikasan algoritam ($\sqrt{1.000.000.000} \approx 30000$). Ulaz od milijardu elemenata obrade za nekoliko milisekundi (npr. Ispitivanje da li je broj prost, faktorizacija broja na proste činioce)
- **Linearna složenost O(n)** – veoma efikasan algoritam u slučaju obrade nizova jer nam je samo za učitavanje i ispis elemenata zahteva to vreme, kada treba da pogledamo ceo ulaz (npr. minimuma/maksimuma serije elemenata)
- **Kvazilinearna složenost O($nlog\ n$)** – malo sporiji od linearnog algoritma, a puno brži od kvadratnog algoritma (npr. efikasno sortiranje, algoritmi zasnovani na tehnici podeli pa vladaj, korišćenje struktura podataka sa logaritamskim vremenom pristupa)
- **Kvadratna složenost O(n^2)** – obično neefikasni algoritmi za obradu nizova (zasnovani na gruboj sili tj. ugnježdenim petljama), ali efikasni za obradu matrica jer već učitavanje i ispis matrice zahteva ovo vreme (npr. sortiranje selekcijom, umetanjem)
    - Za 1s - nekoliko desetina hiljada elemenata
    - Obrada milijardu elemenata praktično nemoguća (zahtevala bi nekoliko desetina godina)
- **Kubna složenost O(n^3)** – izuzetno neefikasni algoritmi za obradu nizova, obično zasnovani na trostrukim ugnježdenim petljama (npr. množenje matrica)
    - Za 1s - najvise 1000 elemenata
    - Obrada million elemenata praktično nemoguća (zahtevala bi nekoliko desetina godina)
- **Eksponencijalna složenost O(2^n)** – izuzetno neefikasan algoritam obično zasnovan na gruboj sili ili iscrpnoj pretrazi (npr. ispitivanje svih varijacija, svih podskupova, svih mogućih puteva kroz drvo ili graf)
    - Za 1s - ispod 30 elemenata
    - Obrada već 60 elemenata je praktično nemoguća (zahtevala bi preko 30 godina)
- **Faktorijelna složenost O($n!$)** – još neefikasnije od eksponencijalne složenosti (npr. ispitivanje svih permutacija)
    - Za 1s - 12 do 13 elemenata
    - Obrada već dvadesetak elemenata je praktično nemoguća (zahtevala bi preko 70 godina)

---

6. Složenost nekih čestih oblika petlje

```
for (int i = 0; i < n; i++)
      // kod složenosti O(1)
```

Složenost petlje je O(n).

```
for (int i = m; i < n; i++)
      // kod složenosti O(1)
```

Složenost petlje je O(*n* - *m*).

```
for (int i = 0; i < n; i += 2)
      // kod složenosti O(1)
```

Složenost petlje je O(*n*), petlja se izvršava $\frac{n}{2}$ puta pa se konstantni faktor $\frac{1}{2}$ može zanemariti.

```
for (int i = 0, j = n-1; i < j; i++, j--)
      // kod složenosti O(1)
```

Složenost petlje je O(*n*), pokazivači se susreću približno na sredini opsega, a telo petlje se izvršava oko $\frac{n}{2}$ puta.

```
for (int i = 0; i < m; i++)
      for (int j = 0; j < n; j++)
            // kod složenosti O(1)
```

Složenost petlji je O(*mn*).

```
for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
            // kod složenosti O(1)
```

Složenost petlji je O(n^2).

```
for (int i = 0; i < n; i++)
      for (int j = i+1; j < n; j++)
            // kod složenosti O(1)
```

Složenost petlji je O(n^2) jer je broj izvršavanja tela unutrašnje petlje je $(n - 1) + (n - 2) + \ldots + 2 + 1$ što je jednako $\frac{n(n-1)}{2}$, složenost je kvadratna, a konstantni faktor $\frac{1}{2}$.

```
for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
            for (int k = 0; k < n; k++)
                  // kod složenosti O(1)
```

Složenost petlji je O(n^3).

```
for (int i = 0; i < n; i++)
      for (int j = i+1; j < n; j++)
            for (int k = j+1; k < n; k++)
                  // kod složenosti O(1)
```

Složenost petlji je O(n^3). Svakom izvršavanju tela petlji odgovara jedna tročlana kombinacija elemenata skupa 0, …, n - 1. Takvih kombinacija ima $\binom{n}{3} = \frac{n(n-1)(n-2)}{3 \cdot 2}$, složenost je kubna, a konstantni faktor $\frac{1}{6}$.

```
for (int i = 0; i < m; i++)
      // kod složenosti O(1)

for (int i = 0; i < n; i++)
      // kod složenosti O(1)
```

Složenost je O(*m* + *n*).

```
for (int i = 0; i < n; i++)
      // kod složenosti O(1)
for (int i = 0; i < n; i++)
      // kod složenosti O(1)
```

Složenost je O(*n*) jer se telo prve petlje izvrši *n* puta, a zatim telo drudge *n* puta, pa se tela obe petlje ukupno izvrše *2n* puta, ali to je O(*n*).

```
for (int i = 0; i < n; i++)
      for (int j = i+1; j < n; j++)
            // kod složenosti O(1)

for (int i = 0; i < n; i++)
      // kod složenosti O(1)
```

Složenost je O(n^2) jer se telo ugnježdenih petlji izvršava $\frac{n(n-1)}{2}$ puta, a zatim telo drudge petlje *n* puta što je zanemarivo malo u odnosu na broj izvršavanja tela ugnježdenih petlji.

Broj ugnježdenih petlji ne odgovara uvek stepenu polinoma!

```
for (int i = 0; i*i <= n; i++)
      // kod složenosti O(1)
```

Složenost petlje je O($\sqrt{n}$), a ne O(*n*), jer se petlja izvršava sve dok je $i^2 \leq n$ тј. $i^2 \leq \sqrt{n}$.

```
for (int i = 0; i < n; i *= 2)
      // kod složenosti O(1)
```

Složenost je O($log\ n$) jer se vrednost brojača duplira u svakom koraku.

```
for (int i = 0; i < 10; i++)
      // kod složenosti O(1)
```

Složenost prethodnog koda je O(*1*) jer je broj izvršavanja petlje konstantan i ne zavisi ni od jednog parametra.

```
for (int i = 1; i <= n; i++)
      for (int j = 1; j < i; j *= 2)
            // kod složenosti O(1)
```

Složenost je O($nlog\ n$) jer se telo unutrašnje petlje izvršava ukupno $log\ 1\ +\ log\ 2\ +\ \ldots\ +\ log\ n$ puta, što je manje ili jednako od $log\ n\ +\ log\ n\ +\ \ldots\ +\ log\ n$ jer je svaki sabirak manji ili jednak $log\ n$.

```
for (int i = n; i >= 0; i /= 2)
      for (int j = 1; j < i; j++)
            // kod složenosti O(1)
```

Složenost petlji je O(*n*) jer je broj izvršavanja unutrašnje petlje $n + \frac{n}{2} + \frac{n}{4} + \ . \ . \ .$, što je odozgo ograničeno sa *2n* (geometrijski niz za $b_0 = 1$ i $q = \frac{1}{2}$).

```
for (int i = 0; i < n; i++) {
      for (int j = i+1; j < n && P(a[j]); j++)
            // kod složenosti O(1)
      // kod složenosti O(1)
      i = j;
}
```

Složenost koda je O(*n*). Broj izvršavanja unutrašnje petlje zavisi od stanja niza, pa ne možemo znati unapred taj broj, ali možemo izvršiti amortizovanu analizu i izračunati ukupan broj izvršavanja unutrašnje petlje. Unutrašnja petlja uvek kreće od tekuće vrednosti brojača spoljašnje petlje, a spoljašnja petlja nastavlja tamo gde je unutrašnja stala. To znači da će se telo petlje izvršiti najviše *n* puta. Ovakav kod se može naći u algoritmima koji analiziraju sve serije uzastopnih elemenata niza koji zadovoljavaju neko svojstvo.

```
int j = 0;
for (int i = 0; i < n; i++) {
      while (j < n && P(a[j])) {
            // kod složenosti O(1)
            j++;
      }
      // kod složenosti O(1)
}
```

Složenost koda je O(*n*) jer unutrašnja petlja ne inicijalizuje brojač *j* na 0 već se brojač stalno uvećava u unutrašnjoj petlji, pa je broj izvršavanja ograničen sa *2n*.

```
int l = 0, d = n-1;
while (l < d) {
      do l++; while (l < d && P(a[l]));
      do d-; while (l < d && Q(a[d]));
      if (l < d)
            // kod složenosti O(1)
}
```

Složenost koda je O(*n*). Korišćenjem amortizovane složenosti vidimo da se l samo povećava počevši od početka niza, a d samo smanjuje počevši od kraja niza, sve dok se ne sretnu. To će se desiti u najviše *n* koraka. Sličan kod se može naći u algoritmima particionisanja niza.

---

7. Rekurentne jednačine za analizu složenosti, master teorema

Složenost rekurzivnih funkcija se često može opisati **rekurentnim jednačinama**.

Rekurentne jednačine gde se problem svodi na problem dimenzije za jedan manje od početne:

- $T(n) = T(n-1) + O(1)$, $T(0) = O(1)$ - rešenje: O(*n*), npr. traženje minimuma niza
- $T(n) = T(n-1) + O(log\, n)$, $T(0) = O(1)$ - rešenje: O($nlog\, n$), npr. formiranje balansiranog binarnog drveta
- $T(n) = T(n-1) + O(n)$, $T(0) = O(1)$ - rešenje: O(n^2), npr. sortiranje selekcijom

Rekurentne jednačine gde se problem svodi na dva (ili više) problema dimenzije za jedan ili dva manja od početne:

- $T(n) = 2T(n-1) + O(1)$, $T(0) = O(1)$ - rešenje: O(2^n), npr. Hanojske kule
- $T(n) = T(n-1) + T(n-2) + O(1)$, $T(0) = O(1)$ - rešenje: O(2^n), npr. Fibonačijevi brojevi

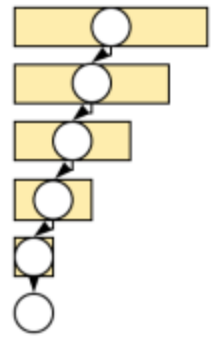
Rekurentne jednačine gde se problem svodi na jedan (ili više) problema značajno manje dimenzije (obično bar duplo manje) od početne:

- $T(n) = T(\frac{n}{2}) + O(1)$, $T(0) = O(1)$ - rešenje: O($log\, n$), npr. binarna pretraga sortiranog niza
- $T(n) = T(\frac{n}{2}) + O(n)$, $T(0) = O(1)$ - rešenje: O(*n*), npr. pronalaženje medijane
- $T(n) = 2T(\frac{n}{2}) + O(1)$, $T(0) = O(1)$ - rešenje: O(*n*), npr. obilazak potpunog binarnog drveta
- $T(n) = 2T(\frac{n}{2}) + O(n)$, $T(0) = O(1)$ - rešenje: O($nlog\, n$), npr. sortiranje objedinjavanjem

Da bismo stekli intuiciju kako se dobijaju ova asimptotska rešenja, možemo primeniti **odmotavanje rekurzije**.

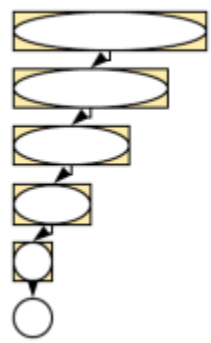
Primer: $T(n) = T(n-1) + O(1)$, $T(0) = O(1)$

$$
\begin{aligned}
T(n) &= T(n-1) + O(1) \\
&= T(n-2) + O(1) + O(1) \\
&= T(n-3) + O(1) + O(1) + O(1) \\
&= \dots \\
&= T(0) + n \cdot O(1) \\
&= O(1) + n \cdot O(1) \\
&= O(n).
\end{aligned}
$$



Primer: $T(n) = T(n-1) + O(n)$, $T(0) = O(1)$

$$
\begin{aligned}
T(n) &= T(n-1) + cn \\
&= T(n-2) + (n-1) + cn \\
&= \dots \\
&= T(0) + c(1 + \dots + n) \\
&= O(1) + cn(n+1)/2,
\end{aligned}
$$



Master teorema:

Rešenje rekurentne jednačine $T(n) = aT(\frac{n}{b}) + O(n^k)$, $T(0) = O(1)$ gde su ***a*** *i* ***b*** celobrojne konstante $a, b \geq 1$ i ***c*** *i* ***k*** pozitivne realne konstante je:

$$
T(n) = \begin{cases} \Theta(n^{\log_b a}) \text{ , ako je } \log_b a > k \\ \Theta(n^k \log n) \text{ , ako je } \log_b a = k \\ \Theta(n^k) \text{ , ako je } \log_b a < k \end{cases}
$$

Primeri:

$T(n) = 2T(\frac{n}{2}) + O(1)$, $T(0) = O(1) \rightarrow a = 2$, $b = 2$, $k = 0 \Rightarrow log_2 2 = 1 > k \Rightarrow \Theta(n)$

$T(n) = 2T(\frac{n}{2}) + O(n)$, $T(0) = O(1) \rightarrow a = 2$, $b = 2$, $k = 1 \Rightarrow log_2 2 = 1 = k \Rightarrow \Theta(nlog\, n)$

$T(n) = T(\frac{n}{2}) + O(n)$, $T(0) = O(1) \rightarrow a = 1$, $b = 2$, $k = 1 \Rightarrow log_2 1 = 0 < k \Rightarrow \Theta(n)$

8. Odsecanje u linearnoj pretrazi – ilustrovanje kroz primere

9. Odsecanje u linearnoj pretrazi – *primeri: prosti brojevi, Eratostenovo sito*

Jedan od osnovnih principa za dobijanje efikasnijih algoritama je da računar ne treba da izračunava stvari za koje može unapred proceniti da nisu potrebne za dobijanje konačnog rešenja. Jedan primer se javlja kod linearne pretrage. Kada preskočimo proveru elemenata za koje unapred znamo da ne mogu da zadovolje uslov pretrage kažemo da smo izvršili **odsecanje u linearnoj pretrazi**. Dakle, ako se unapred može dokazati da neki deo prostora pretrage ne sadrži rešenje ili ne sadrži optimalno rešenje koje se traži, tada se taj deo prostora pretrage ne treba pretraživati

Prilikom odsecanja moramo biti posebno pažljivi kako bismo osigurali korektnost algoritma. Treba da budemo sigurani da u delu prostora pretrage koji smo odsekli zaista nije rešenje problema. Neki od najvažnijih primera primene odsecanja su *binarna pretraga*, *bektreking*, *pretraga u dubinu*, *pretraga u širinu*.

Primer - prost broj: ispitati da li je uneti prirodan broj *n* prost.

- Ukoliko redom proveravamo deljivost svim brojevima iz intervala (1, n) jasno je da će složenost algoritma biti O(n).

```
bool prost(int n) {
      if (n == 1)
            return false;
      for (int i = 2; i < n; i++)
            if (n % i == 0)
                  return false;
      return true;
}
```

- Broj *n* ne može biti deljiv brojevima većim od $\frac{n}{2}$ pa pretragu možemo vršiti samo do $\frac{n}{2}$. Takođe, ako broj *n* nije deljiv sa 2 onda nije ni sa jednim parnim brojem, što daje dodatno odsecanje jer možemo proveravati samo neparne brojeve. Ova odsecanja su suštinski nebitna, jer ne doprinose značajno efikasnosti, unapređen je samo konstantni faktor.

```
bool prost(int n) {
      if (n == 1)
            return false;
      if (n == 2)
            return true;
      if (n % 2 == 0)
            return false;
      for (int i = 3; i <= n/2; i += 2)
            if (n % i == 0)
                  return false;
      return true;
}
```

- Delioci broja se uvek javljaju u paru - ako je broj *n* deljiv sa *d* tako da je $d \geq \sqrt{n}$ tada je deljiv i brojem $\frac{n}{d} \leq \sqrt{n}$. Odnosno, ako broj nema delilaca manjih od $\sqrt{n}$ onda nema ni delilaca koji su veći ili jednaki $\sqrt{n}$, pa je dovoljno ispitivati delioce iz intervala [2, $\sqrt{n}$] što će nam dati složenost O($\sqrt{n}$).

```
bool prost(int n) {
      if (n == 1)
            return false;
      for (int i = 2; i*i < n; i++)
            if (n % i == 0)
                  return false;
      return true
}
```

Moguće je dodatno unaprediti konstantan faktor, ali nam to ne donosi značajna poboljšanja.

Primer - Eratostenovo sito: Ispitati da li su brojevi iz intervala [1, n] prosti.

- U naivnom pristupu bismo redom proveravali umnoške svih brojeva i "precrtavali" ih. Složenost će biti O($nlog\ n$) jer u svakom koraku spoljašnje petlje *d* raste pa imamo sve manje i manje izvršavanja unutrašnje petlje.

```
void Eratosten(vector<bool>& prost, int n) {
      prost.resize(n+1, true);
      prost[1] = false;
      for (int d = 2; d <= n; d++)
            for (int x = 2*d; x <= n; x += d)
                  prost[x] = false;
}
```

- Prvo odsecanje koje možemo napraviti jeste da se unutrašnja petlja izvršava samo ako je *d* prost broj, jer ako nije njegovi umnošci su već precrtani tokom precrtavanja umnožaka nekog od njegovih prostih faktora. Drugo odsecanje dobijamo jer ne moramo unutrašnju petlju počinjati od *2d*, već od d^2 . Manji umnošci su već precrtani jer svi imaju prave faktore manje od *d*. Sada možemo videti da unutrašnja petlja uopšte neće biti izvršena ako je d^2 veće od *n*, pa opet vršimo odsecanje u spoljašnjoj petlji. Sa svim ovim odsecanjima dobićemo dosta brži algoritam složenosti O($nloglogn$).

```
void Eratosten(vector<bool>& prost, int n) {
      prost.resize(n+1, true);
      prost[1] = false;
      for (int d = 2; d*d <= n; d++)
            if (prost[d])
                  for (int x = d*d; x <= n; x += d)
                        prost[x] = false;
}
```

---

10. Inkrementalnost – ilustrovanje kroz primere

Često je potrebno izračunati neku vrednost za različite vrednosti nekog parametra. Izračunavanje je **inkrementalno** ako se rezultujuća vrednost za narednu vrednost parametra izračunava korišćenjem već izračunatih vrednosti za prethodnu ili više prethodnih vrednosti parametra. Ovim izbegavamo izračunavanje istih stvari više puta i poboljšavamo složenost.

Primer: izračunavanje **parcijalnih zbirova (zbirova prefiksa)** elemenata niza

Na primer, ako je dat niz 1, 2, 3, 4, 5, njegovi parcijalni zbirovi su redom 0, 1, 3, 6, 10, 15. Primećujemo da se izračunavanje narednog parcijalnog zbira ne mora vršiti sabiranjem svih elemenata od početka, već se može dobiti sabiranjem prethodnog parcijalnog zbira sa tekućim elementom niza (na primer, zbir 1 + 2 + 3 + 4 = 10, se dobija sabiranjem prethodnog zbira 1 + 2 + 3 = 6 i tekućeg elementa 4). Tada važe:

$$Z_0 = 0$$
$$Z_{k+1} = Z_k + a_k$$

gde je Z_k zbir prvih k elemenata niza. Dobili smo seriju brojeva u kojoj se naredni element izračunava na osnovu prethodnog - **rekurentna serija**. Svaki naredni član se izračunava u složenosti ***O(1)*** pa se izračunavanje svih parcijalnih zbirova niza dužine n vrši u složenosti ***O(n).*** Kada bi se svaki parcijalni zbir računao sabiranjem elemenata niza iz početka, tada bi izračunavanje ***k***-tog zbira bilo složenosti ***O(k)***, a izračunavanje svih zbirova složenosti ***O(***n^2***).***

Slično, i minimum i maksimume možemo izračunavati inkrementalno.

Primer: računanje maksimuma prefiksa

```
int maks = a[0];
cout << maks << '\n';
for (int i = 1; i < a.size(); i++) {
      maks = max(maks, a[i]);
      cout << maks << '\n';
}
```

Rekurentna serija:

$$M_0 = a_0$$
$$M_i = max(M_{i-1}, a_i)$$

---

11. Zamena iteracije formulom – ilustrovanje kroz primere

Zamena iteracije formulom podrazumeva da ne teramo računar da vrši dugotrajna izračunavanja kroz neki iterativni postupak, već, tamo gde je to moguće, uanpred izračunamo rezultat tog iterativnog postupka i u program unesemo formulu za krajnji rezultat.

Primer: računanje sume prvih ***n*** prirodnih brojeva

Sumu prvih ***n*** prirodnih brojeva možemo izračunati iterativno pomoću petljeu složenosti ***O(n)***. Međutim, primenom matematike možemo popraviti složenost. Znamo da Gausova formula glasi: $1 + 2 + \ . \ . \ . \ + n = \frac{n(n+1)}{2}$. Ovime u složenosti ***O(1)*** možemo doći do rezultata.

```
int suma(int n) {
      return (n * (n + 1)) / 2;
}
```

Osnovne matematičke formule:

Aritmetički niz

- **Gausova formula**

$$1 + 2 + \ . \ . \ . \ + n = \frac{n(n+1)}{2}$$

- **K-ti član aritmetičkog niza**

$$a_k = a_0 + k \cdot d$$

- **Zbir aritmetičkog niza**

$$a_0 + (a_0 + d) + (a_0 + 2d) + \ . \ . \ . \ + (a_0 + (n - 1) \cdot d) = \sum_{k=0}^{n-1} (a_0 + k \cdot d) = n \cdot a_0 + d\frac{n(n-1)}{2}$$

Geometrijski niz

- **K-ti član geometrijskog niza**

$$b_k = b_0 \cdot q^k$$

- **Zbir geometrijskog niza**

$$b_0 + b_0 \cdot q + b_0 \cdot q^2 + \ . \ . \ . \ + b_0 \cdot q^{n-1} = \sum_{k=0}^{n-1} b_0 \cdot q^k = b_0 \cdot \frac{1-q^n}{1-q}$$

Stepene sume

- **Zbir kvadrata svih prirodnih brojeva od 1 do n**

$$1^2 + 2^2 + \ . \ . \ . \ + \ n^2 = \frac{n(n+\frac{1}{2})(n+1)}{3} \approx \frac{n^3}{3}$$

- **Zbir kubova svih prirodnih brojeva od 1 do n**

$$1^3 + 2^3 + \ . \ . \ . \ + \ n^3 = (\frac{n(n+1)}{2})^2 \approx \frac{n^4}{4}$$

Kombinatorika

- **Broj načina da se iz skupa od n različitih brojeva izaberu 2 broja**

$$\binom{n}{2} = \frac{n(n-1)}{2}$$

- **Broj načina da se iz skupa od n različitih brojeva izaberu 3 broja**

$$\binom{n}{3} = \frac{n(n-1)(n-2)}{3 \cdot 2}$$

Primer - nedostajući broj: u nizu brojeva od 0 do n jedan broj je izostavljen, pronaći ga efikasno bez pamćenja elemenata niza.

Možemo koristiti linearnu pretragu, ali to dovodi do kvadratne složenosti. Drugi način je korišćenje Gausove formule. Pošto znamo zbir prvih n brojeva i možemo izračunati zbir unetih brojeva, nedostaući broj nalazimo kao razliku ove dve vrednosti. Složenost ovog pristupa će biti linearna.

```
int nedostajuci(int n) {
      int zbir;
      for (int i = 0; i < n; i++) {
            int x;
            cin >> x;
            zbir += x;
      }

      int zbirSvih = (n * (n + 1)) / 2;
      return zbirSvih - zbir;
}
```

Primer - NZD dva broja

Možemo primeniti **Euklidov algoritam zasnovan na oduzimanju**: ako su a i b jednaki tada je i njihov NZD jednak, a u suprotnom se problem svodi na pronalaženje NZD-a njihove razlike i manjeg broja. Ovaj algritam će biti složenosti ***O(max(a, b))***.

```
int nzd(int a, int b) {
      while (a != b) {
            if (a > b)
                  a -= b;
            else
                  b -= a;
      }

      return a;
}
```

Primećujemo da ako imamo veliku razliku među brojevima, imaćemo veći broj “seckanja kvadrata”. Npr. ako imamo brojeve 100 i 18, oduzimaćemo 18 od 100 sve dok ne stignemo do broja 10 (ostatak pri deljenju 100 i 18). Ovo možemo izbeći korišćenjem **Euklidovog algoritma zasnovanog na deljenju**: *NZD(a, 0) = a, NZD(a, b) = NZD(b, a % b)*. Ne moramo da proveravamo koji je od brojeva veći jer je b uvek veci od a % b. Složenost je ***O(***$log\ max(a,\ b)$***).***

```
int nzd(int a, int b) {
      while (b != 0) {
            int ostatak = a % b;
            a = b;
            b = ostatak;
      }
      return a;
}
```

12. Prefiksni zbirovi, razlike susednih elemenata – ilustrovanje kroz primere

Prefiksne zbirove računamo inkrementalno i možemo ih koristiti za računanje zbira elemenata na nekom intervalu. Zbir elemenata u intervalu [a, b] dobijamo kao razliku zbira svih elemenata na intervalima [0, b] i [0, a-1]. Dakle, od datog niza, zbirove prefiksa izračunavamo u linearnoj složenosti i onda u konstantnoj složenosti možemo izračunati zbir na nekom segmentu. Takođe, od niza prefiksa možemo dobiti originalan niz u linearnoj složenosti.

Primer - k upita od kojih svaki traži zbir elemenata na nekom segmentu [a, b]

S obzirom da imamo k upita, neefikasno je da kroz petlju k puta računamo zbir elemenata od a do b jer to dovodi do složenosti ***O(n·k)*** gde je n dužina niza. Umesto toga, računamo prefiksne sume i onda koristimo svojstvo iznad. Složenost je sada ***O(n + k)*** jer nam za računanje prefiksnih suma treba ***O(n)*** koraka, a za upite dodatnih ***O(k)*** koraka.

```
int main() {
      int n;
      cin >> n;
      vector<int> a(n);
      for (int i = 0; i < n; i++)
            cin >> a[i];

      vector<int> prefiksi(n+1, 0);
      for (int i = 0; i < n; i++)
            prefiksi[i+1] = prefiksi[i] + a[i];

      int k;
      cin >> k;
      while (k--) {
            int a, b;
            cin >> a >> b;
            cout << prefiksi[b+1] - prefiksi[a] << '\n';
      }
}
```

Tehnika suprotna zbirovima prefiksa jeste tehnika **razlike susednih elemenata.** Ovaj pristup nam omogućava efikasno menjanje čitavih segmenata niza za neku fiksnu vrednost. Povratak na originalni niz dobijamo u linearnoj složenosti računanjem zbirova prefiksa niza razlika.

Primer - uvećavanje segmenta

Ako rešavamo grubom silom (u petlji uvećavamo elemente na pozicijama od p do q) dolazimo do složenosti ***O(n·k)***, gde je n broj elemenata niza i k broj uvećavanja. Međutim, ukoliko umesto niza čuvamo niz razlika susednih elemenata, kada se u nizu povećaju svi elementi od p do q za 1, u nizu razlika se razlika na poziciji p poveća za 1, od pozicija p+1 do q se ne menjaju i razlika na poziciji q+1 se umanji za 1. Od niza razlika do krajnjeg stanja niza dolazimo na gore opisan način. Na ovaj način dolazimo do složenosti ***O(n + k).***

```
int main() {
      int n;
      cin >> n;
      vector<int> razlike(n, 0);
      int p, q;
      while (cin >> p >> q) {
            razlike[p]++;
            if (q < n -1)
                  razlike[q+1]--;
      }
      int x = 0;
      for (int i = 0; i < n; i++) {
            x += razlike[i];
            cout << x << '\n';
      }
}
```

---

13. Primene sortiranja – ilustrovanje kroz primere

Sortiranje podrazumeva ređanje elemenata u odnosu na neki poredak (brojevi po numeričkoj vrednosti, niske leksikografski,..). Jedan je od najznačajnijih problema u računarstvu i postoje razni algoritmi za sortiranje (**QuickSort, MergeSort, HeapSort, SelectionSort, InsertionSort, BubbleSort, ShellSort,..**).

Biblioteke savremenih programskih jezika nude gotove implementacije algoritama za sortiranje i uvek je poželjnije koristiti ih umesto ručnih implementacija (čitljiviji kod, funkcije su pažljivo testirane i potpuno korektne, implementirane su na najefikasniji mogući način). Složenost ovih funkcija je **kvazilinearna - *O(*$n \log n$*).*** Bibliotečke funkcije takođe dopuštaju navođenje različitih kriterijuma sortiranja zadavanjem funkcija poređenja. Sortiranje pomaže u **obradi duplikata** i takođe **grupiše elemente bliske po vrednosti** pa je i njih lakše obrađivati.

Primer - iz niza brojeva efikasno izbaciti duplikate

Nakon učitavanja i sortiranja niza prolazimo kroz sve elemente počevši od drugog (prvi element niza ne može biti duplikat) i dodajemo ih u rezultujući niz ako nisu isti kao njihov prethodnik. Na kraju resize-ujemo niz na onoliko elemenata koliko je ostalo - k. Složenost je ***O(***$nlog\ n$***)***.

```
void izbaciDuplikate(vector<int>& a) {
      sort(begin(a), end(a));
      int k = 1;
      for (int i = 1; i < n; i++)
            if (a[i] != a[i-1])
                  a[k++] = a[i];
      a.resize(k);
}
```

Ovo smo mogli da uradimo i korišćenjem bibliotečke funkcije ***unique.*** Ona organizuje niz tako da su na početku elementi koji nisu duplikati i vraća iterator na poziciju iza kraja tog dela. Na kraju možemo samo izbrisati preostale element od te pozicije do kraja niza funkcijom ***erase*** i dobijamo niz bez duplikata. Složenost ovog pristupa je takođe ***O(***$nlog\ n$***)*** jer je složenost ove dve funkcije ***O(n)***, pa sortiranje preovlađuje.

---

14. Oblici binarne pretrage – ilustrovanje kroz primere

Ukoliko imamo sortiran niz, **binarna pretraga** nam omogućava pronalaženje elements u složenosti ***O(***$log\ n$***)***, što je značajno bolje od linearne pretrage i složenosti ***O(n).*** Za binarnu pretragu možemo ručno implementirati funkcije ili koristiti neku od bibliotečkih funkcija:

- **binary_search(begin(a), end(a), x)** - proverava da li je element x unutar sortiranog niza *a* i vraca bool (true/false)
- **equal_range(begin(a), end(a), x)** - vraća par iteratora koji ograničavaju raspon elemenata koji su jednaki x u sortiranom nizu a
- **lower_bound(begin(a), end(a), x)** - vraća iterator na prvi element koji je veći ili jednak x u sortiranom nizu a
- **upper_bound(begin(a), end(a), x)** - vraća iterator na prvi element koji je storgo veći od x u sortiranom nizu a

Traženje elements u nizu

Algoritam se zasniva na polovljenju intervala prilikom pretrage. Traženi element se na početku poredi sa središnjim i ako je manji od njega, zbog toga što je niz sortiran, znamo da je manji i od svih elemenata desno od središnjeg, pa taj deo možemo odbaciti tj. primenjujemo tehniku odsecanja. Analogno, ako je traženi element veći od središnjeg, odbacujemo levu polovinu. Ako

nije ni manji ni veći, onda je taj element jednak središnjem i tu je kraj pretrage - pronašli smo element. Za pretragu možemo koristiti ugrađenu funkciju ***binary_search*** ili je ručno implementirati.

Primer: ispitati da li se element x nalazi u nizu

```
bool binarnaPretraga(vector<int>& a, int n, int x) {
      int l = 0, d = n-1;
      while (l <= d) {
            int s = l + (d - l) / 2;
            if (x < a[s])
                  d = s - 1;
            else if (x > a[s])
                  l = s + 1;
            else
                  return true;
      }
      return false;
}
```

Traženje prelomne tačke

Ukoliko imamo niz čiji su elementi sortirani po tome da li zadovoljavaju neko svojstvo P, možemo razmatrati pronalaženje **prelomne tačke niza.** Ako u nizu imamo prvo one elemente koji ispunjavaju svojstvo P, a zatim one koji ne ispunjavaju, prelomnom tačkom možemo smatrati poziciju poslednjeg elementa koji ispunjava svojtvo P ili poziciju prvog elements koji ne ispunjava svojstvo P.

Prelomnu tačku možemo lako pronaći korišćenjem funkcija ***lower_bound*** i ***upper_bound*** ili ručnom implementacijom binarne pretrage. Održavaćemo dve pozicije `l` i `d` i invarijantu da svi elementi levo od `l` zadovoljavaju P, a da svi desno od `d` ne zadovoljavaju P. Za elemente na pozicijama [`l`, `d`] smatramo da još ne znamo kojoj grupi pripadaju. Slično kao i u klasičnoj binarnoj pretrazi uzimamo element na središnjoj poziciji između `l` i `d` i proveravamo da li on ispunjava uslov P. Ako da, onda i svi elementi pre njega zadovoljaju P pa granicu `l` pomeramo na s + 1. Ako ne, onda ni ostali elementi posle njega ne zadovoljaju P pa granicu `d` pomeramo na s - 1. Pretragu vršimo sve dok imamo nepoznatih elemenata, tj. dok je `l` ≤ `d` i kada završimo poslednji element koji zadovoljava P biće na poziciji `d` , a prvi koji ne zadovoljava P biće na poziciji `l`.

Primer: naći prvi element koji je veći ili jednak vrednosti x u sortiranom nizu a

```
int veciIliJednak(vector<int>& a, int n, int x) {
      int l = 0, d = n - 1;
      while (l <= d) {
            int s = l + (d - l) / 2;
            if (x > a[s])
                  l = s + 1
            else
                  d = s - 1;
      }
      return a[l];
}
```

Pronalaženje optimalne vrednosti

Binarna pretraga se može upotrebiti i u procesu optimizacije, ako se problem može formulisati kao problem pronalaženja prelomne tačke. Ovaj oblik binarne pretrage se još naziva **binarna pretraga po rešenju.** Binarnu pretragu je moguće primeniti ako problem zadovoljava svojstvo monotonosti. Karakteristično za ovu upotrebu binarne pretrage je to što vrednosti o kojima je reč obično nisu indeksi elemenata niza i obično se umesto korišćenja bibliotečkih funkcija binarna pretraga ručno implementira.

Primer - drva: Drvoseča treba da naseče određenu količinu drveta testerom koja može da se podesi na određenu visinu. Odrediti najvišu visinu testere tako da drvoseče iseče dovoljnu količinu drveta.

Primetimo da važi **svojstvo monotonosti**: ako je $v_1 < v_2$ i ako je za visinu testere v_2 nasečeno dovoljno drva, tada je nasečeno dovoljno drva i za visinu v_1. Za fiksiranu visinu testere u jednom prolasku kroz niz možemo izračunati koliko je drveta nasečeno i da li je to dovoljno - složenost ***O(n).*** Kao granice uzećemo visinu 0 za koju je sigurno nasečeno dovoljno drveta i visinu h_{max} (visina najvišeg drveta) za koju sigurno nije nasečeno dovoljno drveta. Ukupna složenost će biti ***O(***$n \log h_{max}$***).***

```
int drva(vector<int>& visine, int potrebno) {
      int l = 0, d = *max_element(begin(visine), end(visine));
      while (l <= d) {
            int s = l + (d - l) / 2;
            int naseceno = 0;
            for (int visina : visine)
                  if(visina >= s)
                        naseceno += visina - s;
            if (naseceno >= potrebno)
                  l = s + 1;
            else
                  d = s - 1;
      }
      return d;
}
```

---

15. Tehnika dva pokazivača – ilustrovanje kroz primere

Tehnika dva pokazivača obuhvata klasu efikasnih algoritama koje karakteriše postojanje dve ili više brojačkih promenljivih koje se kreću kroz elemente niza (najčešće sortiranog). Ono što ih razlikuje od ugnježdenih petlji jeste to što se brojači uvek kreću u istom smeru, tj. vrednost im se ili stalno povećava ili stalno smanjuje (a ponekad se jedan povećava a drugi smanjuje dok se ne sretnu u sredini).
Realizacija može biti preko jedne petlje koja kontroliše vrednosti obe promenljive ili preko ugnježdenih petlji ali tako da se nakon završetka unutrašnje petlje, spoljašnja promenjiva uvećava do mesta gde se unutrašnja završila. Pošto se svaka promenljiva može promeniti najviše ***n*** puta gde je ***n*** dužina niza ukupno imamo najviše ***2n*** promena pa dobijamo linearnu složenost za razliku od klasičnih ugnježdenih petlji gde bismo imali kvadratnu složenost.

Primer: objedinjavanje elemenata dva sortirana niza u jedan sortiran niz

Održavaćemo dva brojača koja će prolaziti kroz početne nizove i brojač za rezultujući niz. U svakom koraku dodavaćemo manji od dva trenutna elementa nizova sve dok ne stignemo do kraja jednog od njih. Kada se to desi, sve elemente dužeg niza ćemo prebaciti u rezultujući niz. Ne znamo koji će od njih biti duži, ali nije ni potrebno to da znamo. Na kraju ćemo posebnim petljama prekopirati elemente koji su ostali u datim nizovima. Složenost algoritma će biti linearna jer se tačno jednom prolazi kroz svakih od elemenata početnih nizova - ***O(m + n)***.

```
vector<int> objedinjavanje(vector<int>& a, vector<int>& b) {
      int m = a.size(), n = b.size();
      vector<int> c(m + n);

      int i = 0, j = 0, k = 0;
      while (i < m && j < n) {
            c[k++] = a[i] < b[j] ? a[i++] : b[j++];
      }
      while (i < m)
            c[k++] = a[i++];
      while (j < n)
            c[k++] = b[j++];

      return c;
}
```

Primer: određivanje unije dva skupa predstavljena pomoću sortiranih nizova
Slično kao objedinjavanje ali ne znamo unapred koliko će unija imati elemenata pa koristimo **push_back** za dodavanje i obraćamo pažnju da ne dodajemo iste elemente više puta.

```
vector<int> unija(vector<int>& a, vector<int>& b) {
      int m = a.size(), n = b.size();
      vector<int> c;
      c.reserve(m + n);

      int i = 0, j = 0;
      while (i < m && j < n) {
            if (a[i] < b[j])
                  c.push_back(a[i++]);
            else if (a[i] > b[j])
                  c.push_back(b[j++]);
            else {
                  c.push_back(a[i]);
                  i++; j++;
            }
       }
       while (i < m)
            c.push_back(a[i++]);
       while (j < n)
            c.push_back(b[j++]);
       return c;
}
```

Primer: određivanje preseka dva skupa predstavljena pomoću sortiranih nizova

U slučaju preseka dodajemo samo elemente ako su jednaki, a inače uvećavamo odgovarajući pokazivač. Na kraju ne dodajemo elemente koji su višak jer oni nikako ne mogu biti istovremeno u oba niza, tj. ne mogu biti deo preseka.

```
vector<int> presek(vector<int>& a, vector<int>& b) {
      int m = a.size(), n = b.size();
      vector<int> c;
      c.reserve(m + n);

      int i = 0, j = 0;
      while (i < m && j < n) {
            if (a[i] < b[j])
                  i++;
            else if (a[i] > b[j])
                  j++;
            else {
                  c.push_back(a[i]);
                  i++; j++;
            }
       }
       return c;
}
```

---

16. Strukture podataka – klasifikacija, primeri upotrebe

Efikasna oragnizacija podataka u memoriji predstavlja važan preduslov pisanja efikasnih programa. Pored primitivnih tipova podataka savremeni programski jezici nude veliki broj bibliotečkih struktura podataka. **Strukture podataka** možemo posmatrati preko njihovih operacija, tj. funkcionalnosti koje pružaju pa je bitno poznavati njihov apstraktni interfejs ali imati i osećaj o složenosti operacija.

Po načinu implementacije, strukture podataka se mogu grupisati na:

- sekvencijalne kontejnere: **array, string, vector, list, forward_list, deque**
- adaptor kontejnera: **stack, queue, priority_queue**
- asocijativne kontejnere: **set, multiset, map, multimap**

- neuređene asocijativne kontejnere: **unordered_set, unordered_multiset, unordered_map, unordered_multimap**

Primeri upotrebe:

Sekvencijalne kontejnere koristimo kad god hoćemo da skladištimo neku seriju elemenata. Adaptore kontejnera koristimo kad god želimo da se elementi dodaju i uklanjaju na unapred određen način (**stack** - LIFO, **queue** - FIFO, **priority_queue** - dodaju proizvoljno, uklanjaju nerastuće). Skupove možemo koristiti za obradu duplikata, sortiranje,... Mape možemo koristiti za prebrojavanje (broj pojavljivanja slova, broj pojavljivanja reči,...)

---

17. Strukture podatak sa sekvencijalnim pristupom

Sekvencijalni kontejneri se koriste za skladištenje serija elemenata. Oni predstavljaju određena uopštenja primitivnih statički alociranih nizova. Karakteriše ih mogućnost obilaska svih elemenata redom (nekad i u oba pravca) kao i mogućnost indeksnog pristupa elementima. Sve osim array (koji je samo omotač primitivnog statičkog niza i čija dimenzija mora biti poznata) su dinamički alocirani i dopuštaju umetanje i brisanje elemenata na početak, sredinu ili kraj pri čemu složenost zavisi od konkretnog kontejnera.

Niz - arraz<T, size> predstavlja samo tanak omotač oko klasičnog statičkog niza čija je dimenzija poznata u trenutku deklaracije. On omogućava da se klasični statički nizovi mogu koristiti na isti način kao i drugi kontejneri (npr. dodela niza nizu, poređenje nizova sa ==).

Vektor - vector<T> je implementiran preko dinamičkog niza koji se realocira po potrebi. Ova implementacija omogućava efikasan pristup, iteraciju u oba smera i dodavanje i skidanje elemenata sa kraja. Može biti neefiksan kada čuva velike objekte zbog česte realokacije.

- **push_back -** dodavanje na kraj niza
- **pop_back -** uklanjanje sa kraja niza

Niska - string je implementiran slično kao vektor uz dodatne optimizacije i operacije specifične za podatke tekstualnog tipa.

Jednostruko povezana lista - forward_list<T> je implementirana preko jednostruko povezane liste. Omogućava dodavanje na kraj i početak, kao i brisanje sa početka u vremenu ***O(1)*** dok brisanje sa kraja zahteva ***O(n)*** jer se mora izmeniti pretposlednji čvor. Umetanje i brisanje ostalih elemenata u listi zahteva vreme ***O(n)***.Gubi se mogućnost efikasnog indeksnog pristupa (***O(n)***).

Dvostruko povezana lista - list<T> je implementirana preko dvostruko povezane liste. Omogućava i dodavanje i brisanje sa kraja i početka u vremenu ***O(1)***. Umetanje i brisanje

ostalih elemenata je takođe u vremenu ***O(1)***. Mana u odnosu na jednostruke liste je to što je potrebno više memorije, kao i to što pojedinačne operacije mogu biti malo sporije jer se ažuriraju dva pokazivača. Prednost je efikasnije brisanje sa kraja i iteracija unazad. Liste se sve manje koriste jer je alokacija čvorova skupa mada prednost u odnosu na dek i nizove je to što mogu da čuvaju i velike podatke jer ne dolazi do realokacije.

Red sa dva kraja - deque<T> dopušta da se elementi dodaju i uzimaju sa bilo kog kraja reda odnosno kombinuje funkcionalnosti i steka i reda. Implementiran je kao specifična struktura podataka koju možemo zamisliti kao vektor u kome se nalaze pokazivači na blokove elemenata fiksne veličine. Zbog toga on nudi efikasno dodavanje i uklanjanje i sa početka i sa kraja ali i iteraciju u oba smera i indeksni pristup. Sve operacije su složenosti ***O(1)***. Jedan od prednosti u odnosu na vector je što se prilikom realokacije ne kopiraju elementi već samo pokazivači na njih, pa se mogu čuvati i veliki objekti.

- **push_front** - dodavanje na početak
- **push_back** - dodavanje na kraj
- **front** - čitanje sa početka pod pretpostavkom da red nije prazan
- **back** - čitanje sa kraja
- **pop_front** - brisanje sa početka
- **pop_back** - brisanje sa kraja
- **empty** - proverava da li je red prazan
- **size** - broj elemenata u redu

---

18. Strukture podataka sa asocijativnim pristupom

Asocijativni kontejneri podrazumevaju da se umetanje, pretraga i brisanje elemenata vrši na osnovu vrednosti, a ne na osnovu pozicije na kojoj se nalaze, tj. elementima se pristupa preko ključa, a ne preko indeksa. Osnovni asocijativni kontejneri su skupovi i mape, tj. rečnici. Asocijativni kontejneri se dele na **uređene** i **neuređene**.

Uređeni asocijativni kontejneri nude dodatne funkcionalnosti kao što je ispis elemenata u traženom poretku, ali su ponekad sporiji od neuređenih. Svi oni pretpostavljaju da se njihovi elementi (skupovi) odnosno ključevi (mape) mogu porediti relacijskim operatorom. Obično su implementirani pomoću *samobalansirajućih uređenih binarnih drveta* i osnovne operacije su im *logaritamske složenosti*. Takođe postoje i multiset (gde je moguće postojanje više istih elemenata u skupu) i multimap (gde jedan ključ može imati više pridruženiih vrednosti).

Neuređeni asocijativni kontejneri su obično implementirani pomoću *heš-tabela* i karakteriše ih *amortizovana konstantna složenost* osnovnih operacija. Oni pretpostavljaju da postoji heš-funkcija hash koja elemente skupa/ključeve mapa slika u celobrojne vrednosti koje određuju njihovu poziciju. Prilikom heširanja moguće su i kolizije, tj. da više elemenata ima iste

heš-kodove, tj. da se slikaju na istu poziciju. Jedan od načina rešavanja kolizija jeste da se na pozicijama čuva lista svih elemenata koji imaju tu vrednost heš-funkcije.

Skupovi pružaju efikasno dodavanje i izbacivanje elemenata kao i proveru da li se element nalazi u skupu. Podržan je kroz dve klase **set** i **unordered_set**. U njima nije dozvoljeno ponavljanje elemenata, pa postoje **multiset** i **unordered_multiset** u kojima jeste.

- **set<T>** je implementiran pomoću uređenog binarnog drveta u čijim se čvorovima nalaze elementi skupa gde nije moguće postojanje više čvorova sa istim vrednostima.
- **multiset<T>** je implementiran pomoću uređenog binarnog drveta u čijim se čvorovima nalaze elementi multiskupa sa dozvoljenim ponavljanjem elemenata ili u čijim čvorovima se nalaze elementi i broj njihovih ponavljanja.
- **unordered_set<T> i unordered_multiset<T>** su implementirani preko heš-tabela.

Operacije su kod **set** i **multiset** složenosti ***O(*$log\ n$*)***, dok je kod **unordered_set** najgora složenost ***O(n)***, a prosečna ***O(1)***:

- **insert** - ubacuje element u skup ako on već nije u skupu
- **erase** - uklanja element u skupu ako je on u skupu
- **find** - vraća iterator na traženi element ili end iterator ako se element ne nalazi u skupu
- **lower_bound i upper_bound** - podržane nad uređenim skupovima

Mape predstavljaju kolekcije podataka u kojima se ključevima pridružuju neke vrednosti. Podržane su kroz klase **map** i **unordered_map**. Ključevi sortirane mape mogu biti samo vrednosti koje se mogu porediti relacijskim operatorom, a ključevi nesortirane mape mogu biti samo vrednosti koje se lako mogu pretvoriti u broj. U njima nije dozvoljeno pridružiti više vrednosti istom ključu, pa postoje **multimap** i **unordered_multimap** u kojima jeste.

- **map<K, V>** je implementiran pomoću uređenog binarnog drveta u čijim se čvorovima nalaze ključevi i pridružene vrednosti gde nije moguće postojanje više vrednosti za isti ključ.
- **multimap<K, V>** je implementiran pomoću uređenog binarnog drveta u čijim se čvorovima nalaze ključevi i pridružene vrednosti gde je moguće postojanje više vrednosti za isti ključ.
- **unordered_map<K, V> i unordered_multimap<K, V>** su implementirani preko heš-tabela.

Operacije:

- **insert** - ubacuje par ključ-vrednost u mapu. Složenost: ***O(*$log\ n$*)***
- **erase** - uklanja par ključ-vrednost iz mapu. Složenost: ***O(*$log\ n$*)***
- **find** - pretraga preko ključa, vraća iterator na traženi element ili end iterator ako element nije u mapi. Složenost: ***O(*$log\ n$*)***

19. Heširanje – primena i implementacija

Heširanje je jedan od osnovnih načina implementacije *neuređenih skupova i neuređenih mapa*. Ove strukture podrazumevaju čuvanje **slogova** koji se mogu pretražiti na osnovu ključa (kod skupova se čuvaju samo ključevi, dok je kod mapa ključevima pridružena i neka vrednost).

Heširanje omogućava efikasno dodavanje novih elemenata, efikasnu pretragu na osnovu ključa, kao i brisanje i izmenu elemenata sa datim ključem. Sa druge strane, heširanje ne omogućava ispis elemenata u uređenom poretku niti pronalaženje ključa najbližeg datom.

Heširanje podrazumeva korišćenje **heš-funkcija** koja svaki ključ preslikava u neki ceo broj koju nazivamo **heš-vrednost** ili **heš-kod**. Svi podaci se čuvaju u **heš-tabeli** i heš-vrednost ključa određuje poziciju elementa u tabeli. Heš-tabela se implementira kao niz koji na jednoj poziciji može čuvati jedan ili više elemenata. Ova mesta u tabeli se ponekad nazivaju i **slotovi** ili **kofe** jer je nekad moguće čuvati i više slogova na jednoj poziciji. U prevodu, ako je tabela dimenzije *n* onda će heš-funkcija slikati vrednosti iz nekog proizvoljno velikog skupa ključeva u mali skup pozicija tabele *[0, n)*.

Najjednostavniji slučaj heširanja jeste onaj gde se može uspostaviti bijekcija između skupa ključeva i dopuštenih pozicija u nizu. Na primer, ako svakom malom slovu engleske abecede želimo da dodelimo vrednost u nizu od 26 elementa heš-funkcija može biti *h(c) = c - 'a'* koja će *a* slikati u *0*, *b* u *1* itd. U stvarnosti je češća situacija kada je broj potencijalnih ključeva dosta veći od dimenzije tabele. Tada postoje **kolizije** odnosno više ključeva će imati istu heš-vrednost. Tada je bitno rešiti dva problema:

- odabir heš-funkcije koja će dovesti do što manje kolizija
- razrešavanje kolizija kada one nastanu.

Na broj kolizija utiče i veličina tabele pa se ona često proširuje kada broj kolizija postane preveliki. Međutim, čak i da zauzmemo praktično celu memoriju, kolizije nastaju već kad je niz gotovo prazan (za milion elemenata, već nakon oko 1000 elemenata (jednog promila) imamo veću verovatnoću da će se kolizije javiti nego da se neće javiti).

20. Stek - operacije, implementacije, ilustrovanje kroz primere

Adaptori kontejnera predstavljaju sloj iznad nekog od postojećih sekvencijalnih kontejnera korišćenjem njih za skladištenje podataka. Adaptori imaju svoj podrazumevni sekvencijalni kontejner koji se može promeniti prilikom deklaracije. U njih spadaju stek, red i red sa prioritetom.

Stek predstavlja kolekciju podataka gde se oni dodaju po **LIFO (last in, first out)** principu, tj. element se može dodati i skinuti samo sa vrha steka. Operacije složenosti O(1), podržane kroz klasu **stack<T>**:

- **push** - postavlja element na vrh steka
- **pop** - skida element sa vrha steka ako stek nije prazan
- **top** - čita element sa vrha steka
- **empty** - proverava da li je stek prazan
- **size** - vraća broj elemenata na steku
- **emplace** - umesto operacije **push** ako se na steku čuvaju parovi ili n-torke

Stek je podrazumevano implementiran preko vektora za skladištenje podataka. Tip *stack<int>* podrazumeva zapravo *stack<int, vector<int>>*, a moguće je implementirati ga preko jednostruko povezane liste (*stack<int, forward_list<int>>*).

Primer: Učitavanje linija do kraja ulaza a potom ispis u obrnutom redosledu

```
void obrniLinije() {
      stack<string> linije;
      string linija;
      while (getline(cin, linija))
            linije.push(linija);
      while (!linije.empty()) {
            cout << linije.top() << '\n';
            linije.pop();
      }
}
```

---

21. Red – operacije, implementacija, ilustrovanje kroz primere

Red - queue<T> implementira funkcije reda korišćenjem ***deka*** za skladištenje podataka. Moguća je implementacija i preko drugih kontejnera (pre svega dvostruko povezana lista, dok bi vektor i jednostruko povezana lista imali veoma lošu složenost). Podaci se dodaju po **FIFO (first in, first out)** principu tj. element se dodaje na kraj reda, a skida sa početka. Sve operacije su složenosti ***O(1).***

- **push** - dodaje element na kraj reda
- **pop** - uklanja element sa početka reda ako red nije prazan
- **front** - čita element sa početka reda
- **empty** - proverava da li je red prazan
- **size** - vraća broj elemenata u redu

- **emplace** - umesto push ako se u redu čuvaju parovi ili n-torke

Primer: Ispisati poslednjih k linija standardnog ulaza

```
void ispisiLinije(int k) {
      queue<string> linije;
      string linija;
      while (getline(cin, linija)) {
            linije.push(linija);
            if (linije.size() > k)
                  linije.pop();
      }
      while (!linije.empty()) {
            cout << linije.front() << '\n';
            linije.pop();
      }
}
```

---

22. Red sa prioritetom – operacije, implementacija, ilustrovanje kroz primere

Red sa prioritetom je vrsta reda u kome elementi imaju pridružen prioritet, dodaju se u red proizvoljno, a uvek se uklanja element sa najvećim prioritetom. Složenost operacija push i pop je ***O(***$log\ n$***)***, a ostale su ***O(1).***

- **push** - dodaje element proizvoljno u red
- **pop** - uklanja element sa najvećim prioritetom iz reda ako nije prazan
- **top** - ćita element sa najvećim prioritetom
- **empty** - proverava da li je red prazan
- **size** - vraća broj elemenata reda

Red sa prioritetom je implementiran korišćenjem vektora za skladištenje podataka (u vektoru su smešteni elementi specijalnog drveta koji se zove **hip**). Na primer, *priority_queue<int>* podrazumeva *priority_queue<int, vector<int>, less<int>>* gde je *less<int>* funkcija za poređenje elemenata i daje <u>opadajuću uređenost</u> elemenata. Moguće je imati red sa rastućim poretkom, ali nije moguće imati oba istovremeno.

Primer: Ispisati najmanjh k brojeva učitanih sa standardnog ulaza

```
void ispisiKNajmanjih(int k) {
      priority_queue<int> red;
      int x;
      while (cin >> x) {
            red.push(x);
            if (red.size() > k)
                  red.pop();
      }
      while (!red.empty()) {
            cout << red.top() << '\n';
            red.pop();
      }
}
```

---

23. Hip – primena i implementacija

Hip (**heap**) je struktura podataka koja se koristi za implementaciju redova sa prioritetom. On podržava sledeće operacije:

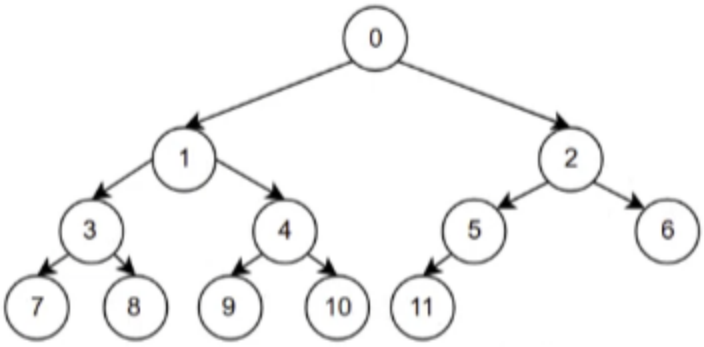
- dodavanje elementa u složenosti ***O(***$log\ n$***)***
- uklanjanje najvećeg elementa u složenosti ***O(***$log\ n$***)***
- čitanje najvećeg elementa u složenosti ***O(1)***

Umesto hipa koji podržava operacije sa najvećim elementom (**maks-hip**) moguće je razmtrati i hip koji podržava operacije sa najmanjim elementom (**min-hip**) ali nikako oba istovremeno.

Elementi hipa se čuvaju u nizu ali zamišljamo da je struktura podataka binarno drvo koje treba da zadovoljava dva uslova:

- **Strukturni uslov** - drvo se popunjava redom, po nivoima i svaki nivo se popunjava redom. vi nivoi osim eventualno posledjeg moraju biti kompletno popunjeni.
- **Uslov rasporeda vrednosti** - u svakom čvoru drveta nalazi se vrednost koja je veća ili jednaka od vrednosti koje se nalaze u njegovim sinovima.

Ukoliko se čvor nalazi na poziciji *k*, njegovi sinovi (ako postoje) nalaze se na pozicijama *2k+1* i *2k+2*. Roditelj čvora na poziciji *m > 0* nalazi se na poziciji $\lfloor \frac{m-1}{2} \rfloor$.



Implementacija operacija:

- **Čitanje najvećeg elementa** - najveći element će se uvek nalaziti u korenu drveta, tj. na početnoj poziciji u nizu. Složenost je ***O(1)***.
- **Dodavanje elementa** - element se prvo doda na kraj niza, da bi bio zadovoljen strukturni uslov hipa. Nakon toga vrednost se *"penje uz hip"*, tj. razmenjuje se sa svojim roditeljem sve dok se ne zadovolji uslov rasporeda vrednosti hipa. Složenost je ***O(***$log\ n$***)*** jer je broj penjanja elementa ograničen visinom drveta, a ona je ograničena logaritmom broja čvorova (jer u potpuno drvo od *k* nivoa staje $2^k - 1$ čvorova).
- **Uklanjanje najvećeg elementa** - iz drveta brišemo poslednji list, a njegovu vrednost upisujemo u koren drveta. Nakon toga vrednost se *"spušta niz hip"*, tj. razmenjuje se sa većim od svojih naslednika sve dok se ne zadovolji uslov rasporeda vrednosti hipa. Složenost je ***O(***$log\ n$***)*** iz istog razloga kao i za dodavanje elementa.

---

24. Podeli-pa-vladaj – ilustrovanje kroz primere

Osnovna ideja tehnike **podeli-pa-vladaj** (**divide-and-conquer**) jeste da je često efikasnije rešavati nekoliko potproblema čija je dimenzija dva ili više puta manja od dimenzije polaznog problema nego rešavati potproblem dimenzije za jedan manje. Kod nekih jednostavnih problema moguće je rešenje dobiti rešavanje samo jednog takvog potproblema, što dovodi do izrazito efikasnih algoritama. Takođe se naziva i tehnika **razlaganja** ili tehnika **dekompozicije**.

Osnovni primeri ove tehnike su sortiranje objedinjavanjem (**MergeSort**) i brzo sortiranje (**QuickSort**), ali i algoritmi obrade binarnog drveta (jer su poddrveta potporblemi koji su u slučaju balansiranih drveta dva puta manji od dimenzije početnog). Kod binarne pretrage se ove tehnika često naziva **smanji-pa-vladaj** (**decrease-and-conquer**) jer se jedan potproblem zanemaruje primenom odsecanja.

Ako su potproblemi koji se rešavaju jednake dimenzije mogu se dobiti neke od sledećih rekurentnih jednačina:

- $T(n) = 2T(\frac{n}{2}) + O(n)$, $T(0) = O(1)$ - rešenje: ***O(***$nlog\ n$***)***
- $T(n) = 2T(\frac{n}{2}) + O(1)$, $T(0) = O(1)$ - rešenje: ***O(n)***
- $T(n) = 2T(\frac{n}{2}) + O(log\ n)$, $T(0) = O(1)$ - rešenje: ***O(n)***
- $T(n) = 2T(\frac{n}{2}) + O(nlog\ n)$, $T(0) = O(1)$ - rešemke: ***O(***$nlog^2 n$***)***

Ako su potproblemi stalno neravnomerne dimenzije moguće je dobiti jednačinu:

- $T(n) = T(n-1) + O(n)$, $T(0) = O(1)$ - rešenje ***O(***n^2***)*** (složenost najgoreg slučaja algoritma quick sort)

U slučaju algoritma smanji-pa-vladaj:

- $T(n) = T(\frac{n}{2}) + O(n)$, $T(0) = O(1)$ - rešenje: ***O(n)***
- $T(n) = T(\frac{n}{2}) + O(n)$, $T(0) = O(1)$ - rešenje: ***O(***$log\ n$***)***

---

25. Podeli-pa-vladaj – primer: brzo sortiranje (QuickSort)

U svakom koraku algoritma se jedan element (**pivot**) dovodi na svoje mesto (poželjno blizu sredine niza). Da bismo problem sveli na sortiranje dva manja podniza potrebno je grupisati elemente tako da su svi elementi niza manji ili jednaki od pivota levo od njega, a svi elementi veći od pivota desno od njega (ako niz sortiramo neopdajuće). Ovaj korak se naziva **particionisanje** i vrši se tehnikom dva pokazivača. Nakon toga, rekurzivno se sortiraju leva i desna polovina niza. Izlaz iz rekurzije predstavlja slučaj kada je niz prazan ili jednočlan. Složenost najgoreg slučaja može biti kvadratna ako pivot stalno deli niz na neravnomerne celine. Prosečna složenost je ***O(***$nlog\ n$***)*** i u praksi ovaj algoritam daje dobre rezultate.

```
void quick_sort(vector<int>& a, int l, ind d) {
      if (l < d) {
            swap(a[l], a[l + rand() % (d - l + 1)]);
            int k = l;
            for (int i = l+1; i <= d; i++)
                  if (a[i] <= a[l])
                        swap(a[i], a[++k]);
```

```
            swap(a[l], a[k]);
            quick_sort(a, l, k - 1);
            quick_sort(a, k + 1, d);
      }
}
```

Ako segment nije neprazan ili jednočlan (`l < d`) za pivot uzimamo proizvoljan element segmenta. Zatim vršimo particionisanje tako da važi da su elementi iz `[l, k]` manji ili jednaki pivotu, elementi iz `(k, i)` su veći od pivota, a elementi iz `[i, d]` su još neispitani. Na kraju razmenimo pivot sa poslednjim manjim ili jednakim elementom i rekurzivno sortiramo levi i desni podniz. Da bismo sortirali niz od *n* elemenata pozivamo funkciju sa `quick_sort(a, 0, n - 1)`

---

26. Podeli-pa-vladaj – primer: sortiranje objedinjavanjem (MeregeSort)

Algoritam deli niz na dva dela čije se dužine razlikuju najviše za 1 (ukoliko niz ima paran broj elemenata onda će podnizovi biti iste dužine). Rekurzivno se sortiraju oba podniza i zatim se sortirane polovine objedinjuju. Objedinjavanje se vrši tehnikom dva pokazivača i potrebno je koristiti pomoćni niz. Izlaz iz rekurzije predstavlja slučaj kada je niz jednočlan (slučaj praznog niza nastupa samo ako je polazni niz prazan).

Složenost najgoreg slučaja je ***O(***$nlog\ n$***)*** što znači da je MergeSort mnogo brži od SelectionSort ili InsertionSort algoritama čija je složenost kvadratna, ali zahteva dodatno ***O(n)*** memorije zbog toga što se elementi čuvaju u pomoćni niz.

```
void merge_sort(vector<int>& a, int l, int d, vecotr<int>& tmp) {
      int s = l + (d - l) / 2;
      merge_sort(a, l, s, tmp);
      merge_sort(a, s+1, d, tmp);

      int i = l, j = s + 1, k = l;
      while (i <= s && j <= d)
            tmp[k++] = a[i] <= a[j] ? a[i++] : a[j++];
      while (i <= s)
            tmp[k++] = a[i++];
      while (j <= d)
            tmp[k++] = a[j++];
      for (int i = l, i <= d; i++)
            a[i] = tmp[i];
}
```

Ako segment nije prazan ili jednočlan (`l < d`) uzimamo sredinu segmenta i rekurzivno sortiramo levi i desni podniz. Nakon toga sortirane podnizove spajamo u pomoćni, a onda elemente pomoćnog kopiram u početni niz. Da bismo sortirali niz od *n* elemenata pozivamo funkciju sa `merge_sort(a, 0, n - 1, tmp)`.

---

27. Pretraga sa povratkom – ilustrovanje kroz primere

Algoritmi grube sile podrazumevaju da se u pretrazi iscrpno nabroje svi kandidati za rešenje i da se za svakog od njih proveri da li zadovoljava uslov ili da li je optimalan.

Algoritmi pretrage sa povratkom (**bektreking**) poboljšavaju tehniku grube sile tako što provere vrše i tokom generisanja kandidata za rešenja i tako što odbacuju parcijalno popunjene kandidate za koje se može unapred utvrditi da se više ne mogu proširiti do ispravnog rešenja. Ovi algoritmi su korisni pri rešavanju **problema zadovoljenja ograničenja** koji podrazumevaju ispitivanje da li postoji kombinatorni objekat koji ispunjava neke uslove, ali i pri rešavanju **problema optimizacije uz ograničenja** koji podrazumevaju pronalaženje najboljeg od svih kombinatornih objekata koji zadovoljavaju neke uslove tj. kombinatornog objekta koji minimalizuje ili maksimalizuje neku datu funkciju. Dakle, umesto da se prvo generišu svi kombinatorni objekti pa da se tek onda proverava da li ispunjavaju neki uslov, bektreking vrši odsecanja ako u nekom trenutku može da ustanovi da data grupa kombinatornih objekata ne može biti rešenje problema.

- Na primer, ako proveravamo da li postoji podskup nekog skupa brojeva čiji je zbir jednak traženom broju, ne moramo grubom silom generisati sve podskupove i proveravati sve zbirove. Ukoliko ustanovimo da je zbir prva dva broja veći od traženog zbira automatski možemo odseći sve podskupove koji sadrže ta dva broja zajedno.

Dodatna optimizacija se može postići kroz **zaključivanje** (**inferrence**). Zaključivanje predstavlja funkciju koja ponekad može da "pogodi" vrednost koja je moguće rešenje problema. Ako je u nekim koracima pretrage zaključivanje previše komplikovano, može i da se preskoči. Algoritam će biti korektan i bez bilo kakvog oblika zaključivanja.

- Na primer, ako prilikom popunjavanja magičnog kvadrata (sve vrste, kolone i obe dijagonale imaju isti zbir) popunimo sve elemente jedne vrste osim jednog, lako možemo da izračunamo vrednost koja mora biti upisana na preostalom mestu.

Kod optimizacionih problema odsecanje može da nastupi i kada se proceni da u delu drveta koje se trenutno pretražuje ne postoji rešenje koje je bolje od najboljeg trenutno pronađenog rešenja. Drugim rečima, rešenja pronađena u dosadašnjem delu pretrage koriste se da bi se

odredile granice na osnovu kojih se vrši odsecanje u drugim delovima pretrage. Ovaj oblik optimizacije naziva se **grananje sa ograničavanjem** (**branch and bound**).

Efikasnost algoritama zasnovanih na bektrekingu uveliko zavisi od kvaliteta odsecanja. Složenost najgoreg slučaja obično ostaje eksponencijalna (što je složenost pretrage grubom silom), ali pažljivo odabrani kriterijumi odsecanja mogu odseći jako velike delove pretrage i značajno ubrzati proces pretrage.

---

28. Generisanje kombinatornih objekata – ilustrovanje kroz primere

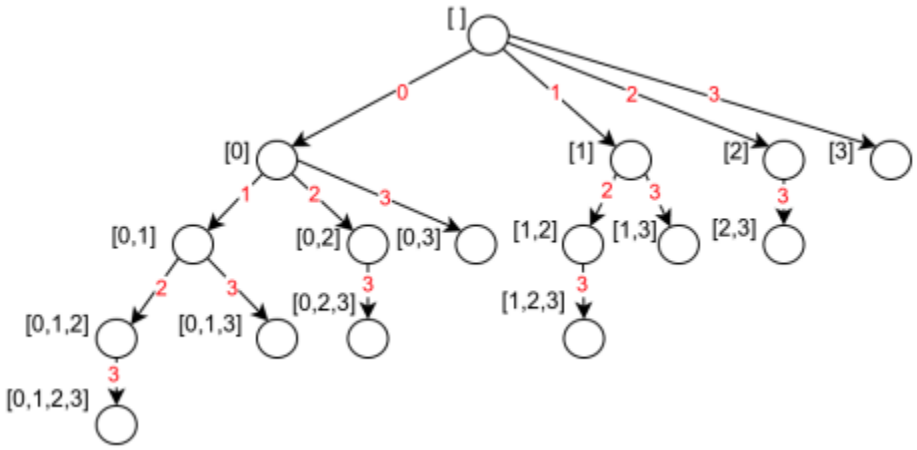
U velikom broju slučaja, prostor potencijalnih rešenja nekog problema predstavlja prostor određenih **kombinatornih objekata** kao što su svi podskupovi nekog konačnog skupa, sve varijacije sa ili bez ponavljanja, sve kombinacije sa ili bez ponavljanja, sve permutacije, sve particije,...

Ovi objekti se obično predstavljaju različitim n-torkama brojeva pri čemu isti objekti mogu da se predstave na različite načine.

- Na primer, podskup skupa {a, b, c, d} može biti predstavljen sortiranim nizom svojih elemenata (npr. [a, c, d]), ali može biti i predstavljen nizom logičkih vrednosti (npr. [1, 0, 1, 1]) ili nizom bitova (npr. 1011). Objekti se tada mogu porediti (najčešće leksikografski).

Svi objekti se takođe mogu predstaviti drvetom. Ono se ne pravi eksplicitno u memoriji, ali nam pomaže da razumemo i osmislimo proces generisanja tj. obilaska objekata.

- Na primer, svi podskupovi četvoročlanog skupa (podskupovi su u čvorovima)



Postoje dva osnovna pristupa generisanja kombinatornih objekata:

- rekurzivno generisanje svih objekata
- generisanje narednog objekta na osnovu prethodnog (najčešće leksikografski)

---

29. Dinamičko programiranje – pojam, ilustrovanje kroz primere

U praksi prilikom izvršavanja rekurzivne funkcije često dolazi do **preklapanja reukrzivnih poziva**, tj. identični rekurzivni pozivi se izvršavaju više puta. Ako se to dešava često program postaje neefikasan (često eksponencijalne složenosti). Tehnika **dinamičkog programiranja** pruža efikasnije rešenje problema tako što vremensku složenost popravlja angažovanjem dodatne memorije u kojoj se beleže rezultati izvršenih rekurzivnih poziva. Dinamičko programiranje dolazi u dva oblika:

- **Memoizacija** ili **dinamičko programiranje naniže** zadržava rekurzivnu definiciju ali u dodatnoj strukturi podataka (najčešće u nizu ili matrici) beleži rezultate rekurzivnih poziva da bi u narednim pozivima u kojima su parametri isti samo očitala iz te strukture.
- **Dinamičko programiranje naviše** u potpunosti uklanja rekurziju (koja se zamenjuje induktivnom konstrukcijom) i pomoćnu strukturu podataka popunjava iscprno u nekom sistematičnom redosledu.

Kod dinamičkog programiranja naviše jasno je da se izračunavaju vrednosti funkcije za sve moguće parametre manje od traženog, dok se kod memoizacije može desiti da se funkcija ne poziva za neke parametre. Iz tog razloga možemo pomisliti da je memoizacija efikasnija ali u praksi je uglavnom potrebno računati vrednost rekurzivne funkcije za veoma veliki broj različitih parametara pa se prednost memoizacije retko sreće.

Primer: Fibonačijev niz

- Klasična rekurzivna implementacija već pri računanju 50. člana Fibonačijevog niza radi veoma sporo jer se funkcija za iste parametre poziva veliki broj puta. Složenost je eksponencijalna.

```
long long fibonaci(int n) {
      if (n == 0 || n == 1)
            return 1;
      return fibonaci(n-1) + fibonaci(n-2)
}
```

- Tehnikom dinamičkog programiranja naniže (memoizacijom) kroz globalni niz čuvamo vrednosti već izračunatih rekurzivnih poziva i na početku svakog poziva proveravamo da li tu vrednost imamo već sačuvanu.
  Potrebno je nekako označiti vrednosti parametara u nizu za koje još ne znamo rezultate rekurzivnih poziva. U slučaju da unapred znamo da su npr. sva rešenja nenegativna, možemo koristiti -1 ili ako su pozitivna 0. Ako nemamo takvih pretpostavki, možemo napraviti dodatni niz logičkih vrednosti kojima čemo ekspicitno kodirati da li za neki parametar znamo ili ne znamo vrednost.
  Za niz *memo* od *n* elemenata treba alocirati *n+1* element. Složenost sada više nije eksponencijalna, već linearna.

```
vector<long long> memo(n+1, -1);

long long fibonaci(int n) {
      if (memo[n] != -1)
            return memo[n];
      if (n == 0 || n == 1)
            return memo[n] = 1;
      return memo[n] = fibonaci(n-1) + fibonaci(n-2)
}
```

- Tehnikom dinamičkog programiranja naviše potpuno eliminišemo rekurziju i sami popunjavamo pomoćnu strukturu od početka do kraja za sve potrebne vrednosti. Krajnja vrednost će na kraju biti na poziciji *n* pomoćne strukture.

```
long long fibonaci(int n) {
      vecotr<long long> dp(n+1);
      dp[0] = dp[1] = 1;
      for (int i = 2; i <= n; i++)
            dp[i] = dp[i-1] + dp[i-2];
      return dp[n];
}
```

- Kod dinamičkog programiranja naviše možemo izvršiti dodatnu memorijsku optimizaciju jer primećujemo da ne moramo pamtiti sve vrednosti, već samo dva prethodnika.

```
long long fibonaci(int n) {
      long long prethodni = 1, pretprethodni = 1;
      for (int i = 2; i <= n; i++) {
            long long tekuci = prethodni + pretprethodni;
            pretprethodni = prethodni;
```

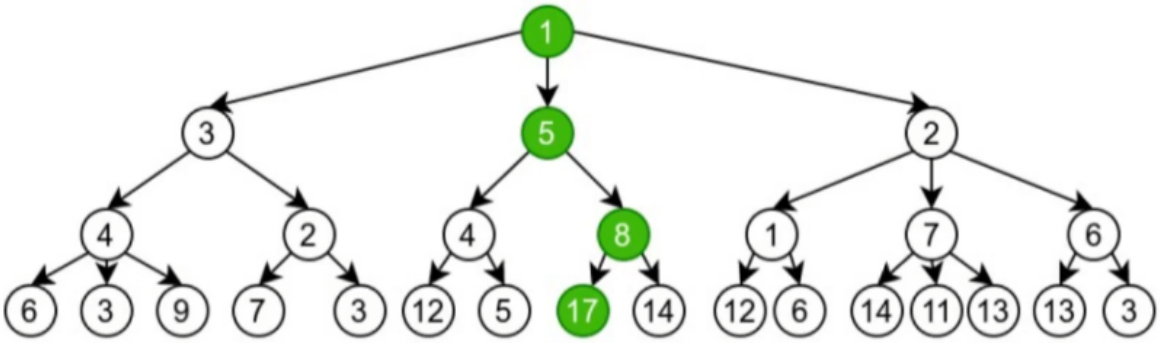
```
            predhodni = tekuci;
}
      return prethodni;
}
```

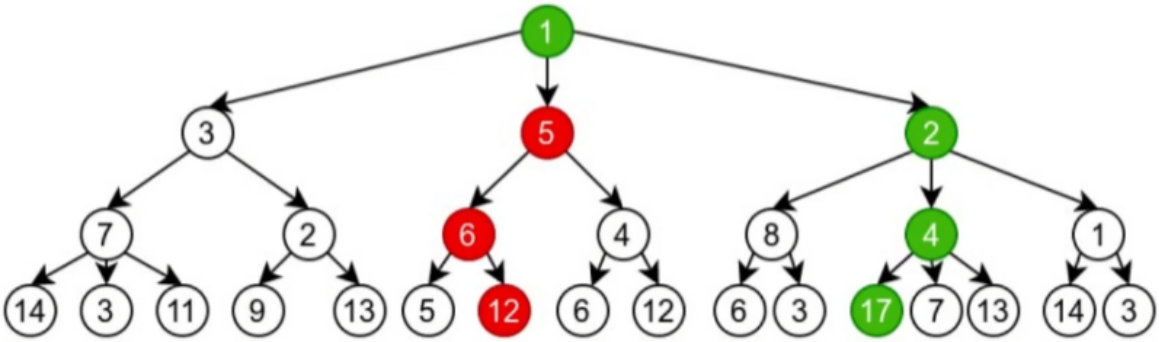
---

30. Pohlepni algoritmi – pojam, ilustrovanje kroz primere

Algoritmi zasnovani na iscrpnoj pretrazi do rešenja dolaze kroz niz koraka gde u svakom koraku analiziraju sve mogućnosti. Algoritmi kod kojih se umesto analize različitih mogućnosti u svakom koraku uzima neko *lokalno optimalno* rešenje nazivaju se **pohlepni** ili **gramzivi** (**greedy**) algoritmi.

Primer: **maksimum drveta pretrage**. U prvom slučaju, biranje lokalnog optimalnog rešenja daje dobar rezultat, a u drugom ne.





Primer: Vraćanje kusura sa što manje novčanica.

- Ako imamo novčanice 1, 2, 5, 10, 20, 50 i 100 i treba da vratimo kusur od 143 dinara, gramziva strategija daje optimalno rešenje - uvek vraćamo najveću novčanicu koja je manja ili jednaka od preostalog iznosa npr. 143 = 100 + 20 + 20 + 2 + 1.
- Ako imamo novčanice 1, 3 i 4 i treba da vratimo kusur od 6 dinara, gramziva strategija daje rešenje 6 = 4 + 1 + 1, ali ono nije optimalno jer kusur možemo vratiti sa dve novčanice (6 = 3 + 3).

Zaključujemo da ove algoritme ima smisla primenjivati samo kod problema kod kojih postoji garancija da će takvi izbori na kraju dovesti do *globalno optimalnog* rešenja. Nekada se primenjuju i kada nemamo garanciju da će rešenje biti optimalno ali garantuju da će dovoljno brzo naći dovoljno kvalitetna suboptimalna rešenja. Ovi algoritmi se nazivaju **gramzive heuristike**.

S obzirom da gramzivi algoritmi ne vrše ispitivanje različitih slučajeva ili iscrpnu pretragu po pravilu su veoma efikasni (u svakom koraku je izvršeno maksimalno moguće odsecanje). Sa druge strane, odsecanje koje se vrši mora biti opravdano i bitno je dokazati korektnost ovih algoritama. Potrebno je dokazati:

- da gramziva strategija uvek daje korektno rešenje, tj. da zadovoljava uslove zadatka
- da gramziva strategija uvek daje optimalno rešenje, tj. da ne postoji ni jedno korektno rešenje koje je bolje. Ovaj uslov je teže dokazati i postoje različite tehnike dokazivanja:
    - **tehnika razmene**: bilo koje optimalno rešenje može se transformisati u rešenje koje daje naša strategija. Na primer, ako imamo dva rasporeda časova koji su optimalani, razmenama časova možemo dobiti raspored koji je dala naša strategija.
    - **tehnika "pohlepno rešenje je uvek ispred"**: rešenje koje daje naša strategija je u svakom koraku bolje (odnosno nije lošije) od ostalih rešenja do kojih se može doći u tom trenutku, tj. pohlepno rešenje nikad ne kasni. Na primer, u svakom koraku prilikom dodavanja časova naš raspored će uvek biti bolji od ostalih rasporeda.
    - **tehnika granice**: odredi se teorijska granica vrednosti optimuma i onda se dokaže da je naše rešenje upravo jednako tom optimumu.

Osnovni nivo – 20 poena

1. Konstrukcija algoritama primenom invarijante petlje – primer: trobojka

```
vodi trobojka(vector<int>& niz, int A, int B) {
      // Invarijanta:
      // 1 1 1 1 1 2 2 2 ? ? ? ? 3 3 3 3
      //           l     i       d
      // [0, l) - 1 grupa (-Inf, A)
      // [l, i) - 2 grupa [A, B]
      // [i, d) - neispitani elementi
      // [d, n) - 3 grupa (B, Inf)
      int l = 0, i = 0, d = niz.size();
      while (i < d) {
            if (niz[i] < A)
                  swap(niz[i++], niz[l++]);
            else if (niz[i] <= B)
                  i++;
            else
                  swap(niz[i], niz[--d]);
      }
}
```

Složenost ovog algoritma je ***O(n)*** jer transformišemo niz u jednom prolasku kroz njega.

---

2. Matematičke osnove izračunavanja složenosti: aritmetički i geometrijski niz, niz stepena

Aritmetički niz

- **Gausova formula**

$$1 + 2 + \ldots + n = \sum_{k=0}^{n} k = \frac{n(n+1)}{2}$$

- **K-ti član aritmetičkog niza**

$$a_k = a_0 + k \cdot d$$

- **Zbir aritmetičkog niza**

$$a_0 + (a_0 + d) + (a_0 + 2d) + \ . \ . \ . \ + (a_0 + (n - 1) \cdot d) = \sum_{k=0}^{n-1} (a_0 + k \cdot d) = n \cdot a_0 + d\frac{n(n-1)}{2}$$

Geometrijski niz

- **K-ti član geometrijskog niza**

$$b_k = b_0 \cdot q^k$$

- **Zbir geometrijskog niza**

$$b_0 + b_0 \cdot q + b_0 \cdot q^2 + \ . \ . \ . \ + b_0 \cdot q^{n-1} = \sum_{k=0}^{n-1} b_0 \cdot q^k = b_0 \cdot \frac{1-q^n}{1-q}$$

- **Zbir geometrijskog reda za** $|q| < 1$

$$\sum_{k=0}^{\infty} b_0 q^k = b_0 \cdot \frac{1}{1-q}$$

- **Za** $b_0 = 1$ **i** $q = 2$ **važi**

$$1 + 2 + \ . \ . \ . \ + 2^{n-1} = 2^n - 1$$

Zato potpuno balansirano drvo sa n nivoa ima $2^n - 1$ čvorova i 2^{n-1} listova

- **Za** $b_0 = 1$ **i** $q = \frac{1}{2}$ **važi**

$$1 + \frac{1}{2} + \frac{1}{4} + \ . \ . \ . \ + (\frac{1}{2})^n = 2 - (\frac{1}{2})^n$$

Stepene sume

- **Zbir kvadrata svih prirodnih brojeva od 1 do n**

$$1^2 + 2^2 + \ . \ . \ . \ + \ n^2 = \sum_{k=1}^{n} k^2 = \frac{n(n+\frac{1}{2})(n+1)}{3} \approx \frac{n^3}{3}$$

- **Zbir kubova svih prirodnih brojeva od 1 do n**

$$1^3 + 2^3 + \ . \ . \ . \ + \ n^3 = \sum_{k=1}^{n} k^3 = (\frac{n(n+1)}{2})^2 \approx \frac{n^4}{4}$$

---

3. Zamena iteracije formulom – primeri: podniske oblika 1…1, rastavljanje na zbir uzastopnih brojeva

Primer: **Broj podstringova koji počinju i završavaju se sa 1**

Naivno rešenje (grubom silom) bi bilo da nabrojimo sve moguće podstringove i da proverimo koliko od njih počinje i završava se jedinicom. Složenost ovog pristupa bi bila ***O(***n^2***).***

```
int broj = 0;
for (int i = 0; i < n; i++)
      for (int j = i+1; j < n; j++)
            if (a[i] == 1 && a[j] == 1)
                  broj++;
```

Mogli bismo tehnikom odsecanja da odsečemo sve one segmente koji ne počinju jedinicom, međutim složenost najgoreg slučaja (sve jedinice) i dalje ostaje ***O(***n^2***).***

```
int broj = 0;
for (int i = 0; i < n; i++)
      if (a[i] == 1)
            for (int j = i+1; j < n; j++)
                  if (a[j] == 1)
                        broj++;
```

Do efikasnog rešenja dolazimo izborom dve jedinice iz skupa svih jedinica (kombinacije). Ako imamo *k* jedinica, dve jedinice možemo izabrati na $\frac{k(k-1)}{2}$ načina. Složenost ovog rešenja je ***O(n)*** jer moramo proći kroz niz kako bismo izračunali broj jedinica.

```
int brojJedinica = 0;
for (int i = 0; i < n; i++)
      if (a[i] == 1)
            brojJedinica++;
int rezultat = brojJedinica * (brojJedinica - 1) / 2;
```

Primer: Rastavljanje na zbir uzastopnih prirodnih brojeva

Jedno rešenje grubom silom bi bilo da proveravamo sve moguće vrednosti za prvi član a_0. Za svaku mogućnost računamo zbir počevši od dvočlanog i uvećavamo broj načina ako je moguće za datu vrednost a_0 dobiti traženi broj. Složenost ovog pristupa je ***O(***$n\sqrt{n}$***).***

```
int brojNacina = 0;
for (int a0 = 1; a0 + (a0 + 1) <= n; a0++) {
      int zbir = a0 + (a0 + 1);
      for (int ai = a0+2; zbir < n; ai++)
            zbir += ai;
      if (zbir == n)
            brojNacina++;
}
```

Drugo rešenje grubom silom podrazumeva da umesto isprobavanja svih mogućnosti za početni član, isprobavamo sve mogućnosti za broj sabiraka *m* (ako je m = 2 to su 1 + 2, 2 + 3, 3 + 4,...). Sada je pitanje kako efikasno za dato *m* i a_0 odrediti traženi zbir. Odgovor je formulom za zbir uzastupnih članova aritmetičkog niza.

$$n = a_0 + (a_0 + 1) + \ldots + (a_0 + (m - 1))$$

$$n = m \cdot a_0 + (1 + 2 + \ldots + (m - 1))$$

$$n = m \cdot a_0 + \frac{m \cdot (m-1)}{2}$$

Pošto najveći broj sabiraka imamo kada je $a_0 = 1$, da bi zbir eventualno mogao da bude jednak *n*, mora da važi $m + \frac{m \cdot (m-1)}{2} \leq n$. Složenost je ista kao i u prethodnom primeri i iznosi ***O(***$n\sqrt{n}$***).***

```
int brojNacina = 0;
for (int m = 2; m + m * (m - 1) / 2 <= n; m++) {
      int a0 = 1;
      int zbir = m * a0 + m * (m - 1) / 2;
      while (zbir < n) {
            a0++;
            zbir = m * a0 + m * (m - 1) / 2;
      }
      if (zbir == n)
            brojNacina++;
}
```

Ključna optimizacija nastupa kada primetimo da nam unutrašnja petlja nije potrebna. Nema potrebe isprobavati različite vrednosti za a_0, već se a_0 može izračunati iz prethodne jednačine na osnovu *m* i *n*. Tada je $a_0 = \frac{n - \frac{m \cdot (m-1)}{2}}{m}$. Da bi zbir postojao, ovo mora da bude ceo broj veći ili jednak 1. To možemo proveriti ispitivanjem ostatka pri deljenju $n - \frac{m \cdot (m-1)}{2}$ sa m, jer nam uslov $m \cdot 1 + \frac{m \cdot (m-1)}{2} \leq n$ garantuje da je broj $\frac{n - \frac{m \cdot (m-1)}{2}}{m} \geq 1$. Takođe, redosled provera je veoma važan, jer je jednačina linearna po a_0, a kvadratna po *m*, tako da je za fiksirano *m*, a_0 prilično jednostavno izračunati, dok za fiksirano a_0 nije jednostavno izračunati *m*.

```
int brojNacina = 0;
for (int m = 2; m + m * (m - 1) / 2 <= n; m++) {
      if ((n - m * (m - 1) / 2) % m == 0)
            brojNacina++;
}
```

---

4. Inkrementalnost – primeri: izračunavanje prefiksnih zbirova, maksimuma, sumiranje redova

Primer: izračunavanje **parcijalnih zbirova (zbirova prefiksa)** elemenata niza

Na primer, ako je dat niz 1, 2, 3, 4, 5, njegovi parcijalni zbirovi su redom 0, 1, 3, 6, 10, 15. Primećujemo da se izračunavanje narednog parcijalnog zbira ne mora vršiti sabiranjem svih elemenata od početka, već se može dobiti sabiranjem prethodnog parcijalnog zbira sa tekućim

elementom niza (na primer, zbir 1 + 2 + 3 + 4 = 10, se dobija sabiranjem prethodnog zbira 1 + 2 + 3 = 6 i tekućeg elementa 4). Tada važe:

$$Z_0 = 0$$
$$Z_{k+1} = Z_k + a_k$$

gde je Z_k zbir prvih k elemenata niza. Dobili smo seriju brojeva u kojoj se naredni element izračunava na osnovu prethodnog - **rekurentna serija**. Svaki naredni član se izračunava u složenosti ***O(1)*** pa se izračunavanje svih parcijalnih zbirova niza dužine n vrši u složenosti ***O(n).*** Kada bi se svaki parcijalni zbir računao sabiranjem elemenata niza iz početka, tada bi izračunavanje ***k***-tog zbira bilo složenosti ***O(k)***, a izračunavanje svih zbirova složenosti ***O(n^2).***

Primer: računanje maksimuma prefiksa

Umesto da u ugnježdenim petljama proveravamo sve prefikse i iznova računamo maksimum (složenost ***O(n^2)***), inkrementalno računamo maksimum (složenost ***O(n)***).

```
int maks = a[0];
cout << maks << '\n';
for (int i = 1; i < a.size(); i++) {
      maks = max(maks, a[i]);
      cout << maks << '\n';
}
```

Rekurentna serija:

$$M_0 = a_0$$
$$M_i = max(M_{i-1}, a_i)$$

Primer: Sumiranje redova. Izračunati sumu $e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \ . \ . \ . \ + \frac{x^n}{n!} + \ldots$

Ključna stvar je da ne računamo svaki put svaki član ove sume ispočetka, već inkrementalno izračunavamo svaki sledeći član. U ovom primeru sledeći član dobijamo tako što prethodni pomnožimo sa $\frac{x}{i}$. Korišćenjem inkrementalnosti, složenost se popravlja sa ***O(n^2)*** na ***O(n).***

```
double x = 1.0, zbir = 1.0, clan = 1.0;
for (int i = 1; i <= n; i++) {
      clan *= x / i;
      zbir += clan;
}
```

5. Prefiksni zbirovi – primer: maksimalni zbir segmenta

Rešenje grubom silom podrazumeva izračunavanje zbira svakog segmenta i pronalaženje najvećeg. Složenost ovog pristupa je ***O***(n^3) (svaki put iznova računamo zbir).
Ako primetimo da je zbir segmenta koji počinje na poziciji i i završava se na poziciji j jednak razlici zbira prvih $j + 1$ članova niza i prvih i članova niza, ako unapred znamo zbirove svih prefiksa, onda zbir svakog segmenta možemo izračunati jako brzo. Međutim i dalje razmatramo sve segmente tako da je složenost spala na ***O***(n^2) što i dalje nije optimalno rešenje.

```
int maks = 0;
for (int i = 0; i < n; i++)
      for (int j = 0; j <= i; j++) {
            int zbir = Z[i+1] - Z[j];
            if (zbir > maks)
                  maks = zbir;
}
```

Vidimo da je za svaku vrednost j umanjenik fiksiran (`Z[i+1]`) i znamo da je vrednost ukupnog maksimuma jednaka maksimumu zbirnog maksimuma za svaku kolonu (za sve segmente $[j, i]$ kada je i fiksirano). Pošto je umanjenik fiksiran, maksimalna razlika za svaku kolonu se dobija kada je umanjilac najmanji tj. potrebno je da znamo najmanji zbir prefiksa do tekuće pozicije. Ovo možemo izračunavati inkrementalno na sledeći način: $Min_i = min(Min_{i-1}, Z_i)$. Složenost je sada ***O(n)*** uz napomenu da je dodatna memorijska složenost ***O(n)*** jer smo čuvali zbirove prefiksa i minimume prefiksa u posebne nizove.

```
int maks = 0;
for (int i = 0; i < n; i++) {
      int maks_zbir_i = Z[i+1] - Min[i];
      if (maks_zbir_i > maks)
            maks = maks_zbir_i;
}
```

Moguća je i implementacija bez korišćenja dodatne memorije.

```
int Z = 0, Min = 0, Maks = 0;
for (int i = 0; i < n; i++) {
      Min = min(Min, Z);
      Z += a[i];
      Maks = max(Maks, Z - Min);
}
```

6. Razlike susednih elemenata – primer: uvećanje segmenta

Ako rešavamo grubom silom (u petlji uvećavamo elemente na pozicijama od p do q) dolazimo do složenosti ***O(n·k)***, gde je *n* broj elemenata niza i *k* broj uvećavanja. Međutim, ukoliko umesto niza čuvamo niz razlika susednih elemenata, kada se u nizu povećaju svi elementi od p do q za 1, u nizu razlika se razlika na poziciji p poveća za 1, od pozicija p+1 do q se ne menjaju i razlika na poziciji q+1 se umanji za 1. Od niza razlika do krajnjeg stanja niza dolazimo određivanjem prefiksnih suma niza razlika. Na ovaj način dolazimo do složenosti ***O(n + k).***

```
int main() {
      int n;
      cin >> n;
      vector<int> razlike(n, 0);
      int p, q;
      while (cin >> p >> q) {
            razlike[p]++;
            if (q < n -1)
                  razlike[q+1]--;
      }
      int x = 0;
      for (int i = 0; i < n; i++) {
            x += razlike[i];
            cout << x << '\n';
      }
}
```

7. Primene sortiranja – primeri: obrada duplikata, grupisanje bliskih vrednosti

Primer - iz niza brojeva efikasno izbaciti duplikate

Nakon učitavanja i sortiranja niza prolazimo kroz sve elemente počevši od drugog (prvi element niza ne može biti duplikat) i dodajemo ih u rezultujući niz ako nisu isti kao njihov prethodnik. Na kraju resize-ujemo niz na onoliko elemenata koliko je ostalo - k. Složenost je ***O(***$n \log n$***)***.

```
void izbaciDuplikate(vector<int>& a) {
      sort(begin(a), end(a));
      int k = 1;
      for (int i = 1; i < n; i++)
            if (a[i] != a[i-1])
                  a[k++] = a[i];
      a.resize(k);
}
```

Ovo smo mogli da uradimo i korišćenjem bibliotečke funkcije ***unique.*** Ona organizuje niz tako da su na početku elementi koji nisu duplikati i vraća iterator na poziciju iza kraja tog dela. Na kraju možemo samo izbrisati preostale element od te pozicije do kraja niza funkcijom ***erase*** i dobijamo niz bez duplikata. Složenost ovog pristupa je takođe ***O(***$nlog\ n$***)*** jer je složenost ove dve funkcije ***O(n)***, pa sortiranje preovlađuje.

Primer: Grupisanje bliskih vrednosti - iz datog niza uzimamo *k* elemenata tako da je razlika između najvećeg i najmanjeg minimalna

Grubom silom bismo svaku moguću grupu od *k* elemenata razmatrali ali bi složenost ovoga bila loša i teško je za implementaciju. Ali ako sortiramo niz, elementi koji su bliski po vrednosti će se naći jedan do drugog i dovoljno je razmatrati samo uzastopne grupe od *k* elemenata. Složenost je ***O(***$nlog\ n$***)***.

```
sort(begin(a), end(a));
int minRazlika = a[n-1] - a[0];
for (int i = 0; i <= n - k; i++)
      minRazlika = min(minRazlika, a[i + k - 1] - a[i]);
```

---

8. Primene sortiranja – primeri: svođenje na kanonski oblik, anagrami

Često imamo potrebu da proverimo da li su dva niza elemenata jednaka, ako se zanemari u kom su redosledu elementi navedeni. Klasičan primer ovoga je provera da li se jedna reč može dobiti permutovanjem slova druge (tj. da li su anagrami). Najbolji način da se ovakva jednakost proveri je da se oba niza svedu na neki **kanonski oblik**, koji neće zavisiti od redosleda elemenata niza. Najjednostavniji način da se takav kanonski oblik dobije
je da se elementi niza sortiraju pre poređenja. Drugi način je da se izvrši prebrojavanje svih elemenata i onda su dva niza jednaka ako imaju isti broj svakog elementa.

Primer: Anagrami - proveriti da li su dve reči anagrami (da li se jedna reč može dobiti permutovanjem slova druge reči)

Grubom silom bismo mogli da proverimo sve permutacije jedne reči i uporedimo da li je neka jednaka drugoj reči međutim ovo rešenje je izuzetno neefikasno i ima složenost ***O(n!)***. Bolji način je da sortiramo slova obe reči (time svodimo reči na kanonski oblik) i zatim uporedimo te dve sortirane reči. Ako su jednake, anagrami su. Složenost je ***O(***$nlog\ n$***)***.

```
sort(begin(niska1), end(niska1));
sort(begin(niska2), end(niska2));
if (niska1 == niska2)
      cout << "reci su anagrami\n";
else
      cout << "reci nisu anagrami\n";
```

Ako na primer imamo niz reči i treba da proverimo koliko parova anagrama ima u tom nizu, možemo prvo sortirati sve reči unutar niza, a zatim sortirati i ceo niz da bi se duplikati našli jedan uz drugi i u jednom proslasku možemo odrediti koliko parova anagrama.

---

9. Tehnika dva pokazivača – primeri: objedinjavanje sortiranih nizova

Primer: objedinjavanje elemenata dva sortirana niza u jedan sortiran niz

Održavaćemo dva brojača koja će prolaziti kroz početne nizove i brojač za rezultujući niz. U svakom koraku dodavaćemo manji od dva trenutna elementa nizova sve dok ne stignemo do kraja jednog od njih. Kada se to desi, sve elemente dužeg niza ćemo prebaciti u rezultujući niz. Ne znamo koji će od njih biti duži, ali nije ni potrebno to da znamo. Na kraju ćemo posebnim petljama prekopirati elemente koji su ostali u datim nizovima. Složenost algoritma će biti linearna jer se tačno jednom prolazi kroz svakih od elemenata početnih nizova - ***O(m + n)***.

```
vector<int> objedinjavanje(vector<int>& a, vector<int>& b) {
      int m = a.size(), n = b.size();
      vector<int> c(m + n);

      int i = 0, j = 0, k = 0;
      while (i < m && j < n) {
            c[k++] = a[i] < b[j] ? a[i++] : b[j++];
      }
      while (i < m)
```

```
            c[k++] = a[i++];
      while (j < n)
            c[k++] = b[j++];

      return c;
}
```

Primer: određivanje unije dva skupa predstavljena pomoću sortiranih nizova

Slično kao objedinjavanje ali ne znamo unapred koliko će unija imati elemenata pa koristimo **push_back** za dodavanje i obraćamo pažnju da ne dodajemo iste elemente više puta.

```
vector<int> unija(vector<int>& a, vector<int>& b) {
      int m = a.size(), n = b.size();
      vector<int> c;
      c.reserve(m + n);

      int i = 0, j = 0;
      while (i < m && j < n) {
            if (a[i] < b[j])
                  c.push_back(a[i++]);
            else if (a[i] > b[j])
                  c.push_back(b[j++]);
            else {
                  c.push_back(a[i]);
                  i++; j++;
            }
       }
       while (i < m)
            c.push_back(a[i++]);
       while (j < n)
            c.push_back(b[j++]);
       return c;
}
```

Primer: određivanje preseka dva skupa predstavljena pomoću sortiranih nizova

U slučaju preseka dodajemo samo elemente ako su jednaki, a inače uvećavamo odgovarajući pokazivač. Na kraju ne dodajemo elemente koji su višak jer oni nikako ne mogu biti istovremeno u oba niza, tj. ne mogu biti deo preseka.

```
vector<int> presek(vector<int>& a, vector<int>& b) {
      int m = a.size(), n = b.size();
      vector<int> c;
      c.reserve(m + n);

      int i = 0, j = 0;
      while (i < m && j < n) {
            if (a[i] < b[j])
                  i++;
            else if (a[i] > b[j])
                  j++;
            else {
                  c.push_back(a[i]);
                  i++; j++;
            }
       }
       return c;
}
```

---

10. Tehnika dva pokazivača – primeri: parovi datog zbira i razlike, segmenti datog zbira

Primer: Broj parova datog zbira - koliko parova brojeva u datom nizu imaju zbir jednak datom

Grubom silom ispitujemo sve parove, složenost ***O(***n^2***)***.
Niz bismo mogli da sortiramo i onda za svaki element tražimo binarnom pretragom u delu niza iza njega njegovu dopunu do traženog zbira, složenost ***O(***$nlog\ n$***)*** ua sortiranje i ***O(***$nlog\ n$***)*** za BP.
Tehnikom dva pokazivača takođe prvo sortiramo niz, a onda pokazivače inicijalizujemo na prvi i poslednji element niza i oni se kreću jedan ka drugom dok se ne susretnu. Ako je zbir ta dva elementa manji od traženog, treba uvećati levi pokazivač, ako je veći, smanjujemo desni pokazivač, a ako je jednak pomeramo i levi i desni pokazivač. Složenost ovog algoritma je ***O(*** $nlog\ n$***)*** zbog sortiranja (da je niz bio sortiran na početku, složenost bi bila ***O(n)***).

```
sort(begin(a), end(a));
int l = 0, d = n - 1, br = 0;
while (l < d) {
      if (a[l] + a[d] < zbir)
            l++;
```

```
	else if (a[l] + a[d] > zbir)
		d--;
	else {
		br++;
		l++;
		d--;
	}
}
```

U prethodnom primeru algoritam radi ako se elementi niza ne ponavljaju. Ako se ponavljaju, možemo eksplicitno prebrojati broj ponavljivanja svakog (tako što napravimo pomoćni niz) i brojač uvećavamo za proizvod broja pojavljivanja (ako nam na primer treba zbir 16 i imamo par 7 i 9 pri čemu se 7 pojavljuje dva puta a 9 tri puta, brojač uvećavamo za $2 \cdot 3 = 6$).
Možemo takođe i prebrojavati implicitno tako što svaki put kad pomeramo pokazivače, pomeramo ih do kraja serije uzastopnih pojavljivanja istog elementa i prebrojimo koliko ima pojavljivanja.

Primer: Broj parova date razlike - koliko parova brojeva u datom nizu imaju razliku jednaku datoj

Kao i malopre, moguće je rešenje pomoću ponovljenih binarnih pretraga.
Tehnikom dva pokazivača ponovo sortiramo niz, a onda pokazivače inicijalizujemo na prvi i drugi element niza i oni se kreću u istom smeru dok desni ne pređe granicu niza pri čemu je levi uvek manji od desnog. Uporedimo razliku ova dva elementa sa traženom, i ako je manja, uvećavamo desni pokazivač (umanjenik), ako je veća uvećavamo levi (umanjilac), a ako je jednaka uvećavamo oba pokazivača. Složenost je kao i u prethodnom zadatku.

```
sort(begin(a), end(a));
int l = 0, d = 1, br = 0;
while (d < n) {
	if (a[d] - a[l] < razlika)
		d++;
	else if (a[d] - a[l] > razlika)
		l++;
	else {
		br++;
		l++;
		d++;
	}
}
```

Primer: Broj segmenta datog zbira

Grubom silom bismo izračunavali zbir svakog segmenta i proveravali da li je jednak datom. Čak i kada se zbir računa inkrementalno, ovo rešenje je složenosti ***O***(n^2).

Znamo da se zbirovi segmenata mogu izračunati kao razlika prefiksnih zbirova i onda se ovaj zadatak svodi na pronalaženje parova date razlike uz napomenu da je niz prefiksa sortiran i nema dodatnog sortiranja pa je rešenje ***O(n)***.

Moguće rešenje je i tehnikom pokretnog prozora gde održavamo granice segmenta pokazivačima i inkrementalno računamo zbir segmenta (ako je zbir manji od traženog, pomeramo desnu granicu i na zbir dodajemo taj novi element, ako je zbir veći, od zbira oduzimamo element na koli pokazuje levi pokazivač i uvećavamo ga, a ako je jednak pomeramo oba pokazivača). Složenost je ***O(n)***.

---

11. Stek: operacije, implementacija – osnovni primeri upotrebe (uparenost zagrada, istorija veb-pregledača, push-pop rekonstrukcija)

Primer: Uparenost zagrada - data je niska koja sadrži tri vrste zagrada () [] {}, proveriti da li su one ispravno uparene.

Idemo redom kroz nisku i ako naiđemo na otvorenu zagradu, dodajemo je na stek, a ako naiđemo na zatvorenu zagradu onda proveravamo da li je stek neprazan (ako je prazan, nisu dobro uparene) i da li ona odgovara zagradi na vrhu steka (ako ne odgovara, nisu dobro uparene). Ako su oba ova uslova zadovoljena, onda skidamo poslednju zagradu sa vrha steka i tako nastavljamo do kraja niske. Ako je na kraju stek prazan znači da su zagrade dobro uparene, u suprotnom nisu dobro uparene. Složenost je ***O(n)***.

Primer: Istorija veb-pregledača

Kako nailazimo na adrese, dodajemo ih na stek i ispisujemo, a ako naiđemo na liniju *back* proveravamo da li je stek prazan. Ako jeste, ispisujemo “-”, a ako nije, skidamo poslednju adresu i ponovo proveravamo da li je prazan, ako nije ispisujemo adresu sa vrha, a ako jeste ispisujemo “-”

```
stack<string> linije;
string s;
while (getline(cin, s)) {
      if (s == "back") {
            if (!linije.empty())
```

```
                linije.pop();
            if (!linije.empty())
                cout << linije.top() << '\n';
            else
                cout << "-" << '\n';

        }
        else {
            linije.push(s);
            cout << linije.top() << '\n';
        }
}
```

Primer: Push-pop rekonstrukcija - imamo 2 niza koji predstavljaju operacije push i pop i treba rekonstruisati kojim redom su se operacije pojavljivale

Prolazimo sa 2 pokazivača kroz nizove push i pop.

- Ako je tekući element na vrhu steka jednak tekućem elementu u nizu pop, skidamo ga sa steka i ispisujemo pop
- Ako tekući element na vrhu steka nije jednak tekućem elementu u nizu pop ili ako je stek prazan, proveravamo da li postoji još elemenata u nizu push:
    - Ako postoji, dodajemo sledeći element na stek i ispisujemo push
    - Ako ne postoji onda rešenje nije moguće jer ne možemo izvršiti ni push ni pop i ispisujemo "-"

---

12. Primena skupova i multiskupova – osnovni primeri: duplikati, broj različitih dužina duži, sortiranje (TreeSort)

Primer: Duplikati - učitavamo *n* brojeva i treba odrediti koliko ima različitih
Pošto skupovi ne dopuštaju duplikate, dovoljno je samo ubacivati jedan po jedan element u skup i na kraju ispisati veličinu skupa. Ako koristimo uređene skupove, složenost je ***O(***$n \log n$***)*** jer je operacija dodavanja složenosti ***O(***$\log k$***)*** gde je *k* broj elemenata skupa, a ako koristimo neuređene skupove, složenost je ***O(n)***.

```
set<unsigned> a; //unordered_set<unsigned> a;
for (int i = 0; i < n ; i++) {
      unsigned x; cin >> x;
      a.insert(x);
```

```
}
cout << a.size() << '\n';
```

Primer: Broj različitih dužina duži

Zadatak je sličan prethodnom, u skup ubacujemo sve dužine duži koje računamo po formuli $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}$ i ispisujemo veličinu skupa. Međutim, nikada ne bi trebalo praviti skupove realnih brojeva jer zbog zaokruživanja može doći do toga da se dve jednake duži protumače kao različite. Ovo rešavamo tako što umesto dužina duži koje su realni brojevi, pamtimo kvadrate dužina duži koje su prirodni brojevi. Složenost je ***O(***$nlog\ n$***)***.

Primer: Sortiranje (TreeSort)

Ako elemente ubacujemo redom u uređeni skup, prilikom ispisa elemenata oni će biti sortirani, ali pored toga nećemo imati nijedan duplikat. Ako želimo da implementiramo klasično sortiranje sa ponovljenim elementima, možemo koristiti uređeni multiskup koji podržava ponavljanje elemenata. Ovo se naziva **TreeSort** zato sto je skup implementiran preko uređenog binarnog drveta. Složenost je ***O(***$nlog\ n$***)***.

```
multiset<int> a;
int n; cin >> n;
for (int i = 0; i < n; i++) {
      int x; cin >> x;
      a. insert(x);
}

for (int x : a)
      cout << x << '\n';
```

---

13. Primena mapa – osnovni primeri: prebrojavanje reči, segment datog zbira u nizu celih brojeva, broj segmenata sa različitim elementima

Primer: Prebrojavanje reči - u nizu od *n* reči odrediti koja se pojavljuje najveći broj puta

Čitamo reči sa standardnog ulaza i svakoj reči u mapi pridružujemo broj pojavljivanja te reči. Nakon toga prolazimo kroz mapu i tražimo maksimalan broj pojavljivanja. Ako pretpostavimo da je složenost unošenja i ažuriranja vrednosti u mapi sa *k* elemenata ***O(***$log\ k$***),*** složenost celog algoritma je ***O(***$nlog\ n$***)***.

```
map<string, int> brojPojavljivanja;
string s;
while (cin >> s)
      brojPojavljivanja[s]++;
int max = 0;
string maxRec;
for (auto it : brojPojavljivanja) {
      if (it.second > max) {
            max = it.second;
            maxRec = it.first;
      }
cout << maxRec << " " << max << '\n';
}
```

Primer: Segmenti datog zbira u nizu celih brojeva

Pošto su u pitanju celi brojevi, nemamo monotonost kao kod prirodnih brojea, pa ne možemo koristiti tehniku dva pokazivača.
Ideja: Pošto znamo da je zbir segmenta na pozicijama [i, j] jednak razlici dva prefiksna zbira $z_{j+1} - z_i$, za svaki prefiksni zbir z_{j+1} proveravamo da li pre njega postoji neki zbir z_i takav da je $z_{j+1} - z_i = Z$ tj. da li postoji element $z_{j+1} - Z$. Ako postoji, povećavamo brojač segmenata za vrednost koliko se puta pojavio taj zbir. Ovo čuvamo u mapi i na kraju ažuriramo broj pojavljivanja ovog zbira. Složenost je ***O(***$nlog\ n$***)***.

```
int Z; cin >> Z;
map<int, int> z;
z[0]++;
int tekuci_zbir = 0, brojSegmenata = 0;
for (int i = 0; i < n; i++) {
      tekuci_zbir += a[i];
      if (z.find(tekuci_zbir - Z) != z.end())
            brojSegmenata += z[tekuci_zbir - Z];
      z[tekuci_zbir]++;
}
cout << brojSegmenata << '\n';
```

Primer: Broj segmenata sa različitim elementima

14. Red: operacije, implementacija – osnovni primeri primene (*k* poslednjih linija, segment maksimalnog proseka dužine *k*, maksimalna bijekcija)

Primer: Poslednjih *k* učitanih linija

Možemo učitavati linije jednu po jednu i kad njihov broj prestigne k, izbacujemo prvu učitanu. Jasno je da koristimo red. Složenost ovog algoritma pod pretpostavkom da ćemo učitati *n* linija je ***O(n)*** jer je ubacivanje i izbacivanje iz reda O(1).

```
queue<string> red;
string linija;
while (getline(cin, linija)) {
      red.push(linija);
      if (red.size() > k)
            red.pop();
}

while (!red.empty()) {
      cout << red.front() << '\n';
      red.pop();
}
```

Primer: Segment maksimalnog proseka dužine *k* - među učitanih *n* brojeva odrediti *k* uzastupnih koji imaju najveći prosek

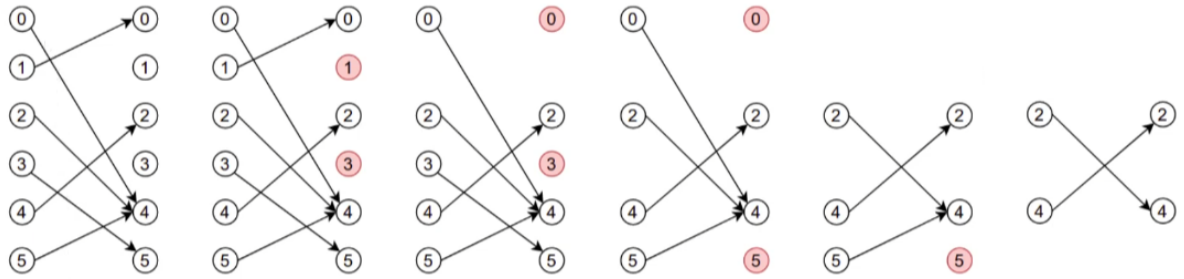
Koristimo tehniku pokretnog prozoro i u redu čuvamo poslednjih *k* elemenata, a prosek možemo predstaviti kao zbir *k* elemenata (prosek bismo dobili deljenjem tog zbira sa *k*) i njega računamo inkrementalno. Složenost je ***O(n),*** ali memorijska složenost je ***O(k)*** što može biti važno ako je *k* dosta manje od *n*.

```
queue<double> red;
double suma = 0.0;
for (int i = 0; i < k; i++) {
      double x; cin >> x;
      red.insert(x);
      suma += x;
}
```

```
int maxPocetak = 0;
double maxSuma = suma;
for (int i = 1; i <= n-k; i++) {
      double x; cin >> x;
      suma = suma - red.front() + x;
      red.insert(x); red.pop();
      if (suma >= maxSuma) {
            maxSuma = suma;
            maxPocetak = i;
      }
}
```

Primer: Maksimalna bijekcija - data je funkcija *f* na nekom skupu i treba odrediti maksimalni podskup tog skupa na kom je funkcija *f* bijekcija

Možemo se opredeliti za uslov injektivnosti ili surjektivnosti. Kada bismo razmatrali injektivnost, posmatrali bismo one čvorove u koje se slika više od jednog originala, međutim tu bi trebalo da isprobavamo puno mogućnosti. Bolje je razmatrati surjektivnost, odnosno one čvorove u koje se ne slika ni jedan original (čiji je ulazni stepen 0). U redu čuvamo sve takve čvorove i izbacujemo ih jedan po jedan, međutim moguće je da se pri izbacivanju nekog čvora pojavi novi čvor u koji se ne slika ni jedan original, pa i njega onda treba izbaciti (ubacujemo ga u red). Kada izbacimo sve takve čvorove, dobijamo maksimalnu bijekciju.



---

15. Red sa dva kraja: operacije, implementacija – osnovni primeri primene

Primer: Ograničena istorija veb-pregledača

Pošto se novi sajtovi dodaju na kraj a skidaju sa početka kada se dodje do ograničenja (onaj najraniji treba zaboraviti), ovde koristimo red sa dva kraja.

```
deque<string> linije;
string s;
while (getline(cin, s)) {
      if (s == "back") {
            if (!linije.empty())
                  linije.pop_back();
            if (!linije.empty())
                  cout << linije.back() << '\n';
            else
                  cout << "-" << '\n';

      }
      else {
            cout << s << '\n';
            if (linije.size() == n)
                  linije.pop_front();
            linije.push_back(s);
      }
}
```

Primer: Sortiranje - svi ispred veći ili svi ispred manji - treba sortirati niz u složenosti ***O(n)*** pri čemu za svaki element važi da su svi elementi ispred njega ili veći ili manji od njega

Ubacujemo prvi element niza u red sa dva kraja. Krećemo se kroz ostale elemente niza i ako je tekući element veći od poslednjeg u nizu, ubacujemo ga na kraj, a ako je manji, ubacujemo ga na početak.

```
deque<int> a;
int x; cin >> x;
a.push_back(x);

for (int i = 1; i < n; i++) {
      int x; cin >> x;
      if (x > a.back())
            a.push_back(x);
      else
            a.push_front(x);
}
```

---

16. Red sa prioritetom: operacije – osnovni primeri primene: *k* najmanjih elemenata, sortiranje

Primer: *k* najmanjih elemenata
U red sa prioritetom najpre ubacujemo *k* elemenata i zatim do kraja niza proveravamo da li je sledeći element manji od najvećeg u redu, ako jeste izbacujemo najveći element iz reda i zamenjujemo ga tekućim elementom niza.

```
priority_queue<int> pq;
for (int i = 0; i < k; i++) {
      int x; cin >> x;
      pq.push(x);
}

for (int i = k; i < n; i++) {
      int x; cin >> x;
      if (x < pq.top()) {
            pq.pop();
            pq.push(x);
      }
}
```

Primer: Sortiranje uz pomoć reda sa prioritetom
Pošto je red sa prioritetom implementiran obično pomoću hipa, ovde se koristi algoritam **HeapSort**. Algoritam hip sort koristi činjenicu da je određivanje i uklanjanje najmanjeg elementa iz reda sa prioritetom prilično efikasna operacija. Stoga se sortiranje može realizovati tako što se svi elementi umetnu u red sa prioritetom (implementiran pomoću strukture hip), iz koga se zatim pronalazi i uklanja jedan po jedan najmanji element (pošto se uklanjaju najmanju a ne najveći elementi, koristimo greater<int>). Složenost dodavanja i izbacivanja elemenata u red sa prioritetom je složenosti ***O(***$log\ k$***)***, gde je *k* broj elemenata u redu sa prioritetom, ukupna složenost algoritma je ***O(***$nlog\ n$***)***.

```
priority_queue<int, vector<int>, greater<int>> pq;
for (int i = 0; i < n; i++) {
      int x; cin >> x;
      pq.push(x);
}
while (!pq.empty()) {
      cout << pq.top() << '\n';
      pq.pop();
}
```

17. Heširanje – razrešavanje kolizija ulančavanjem

Kolizije nastaju kada više ključeva ima istu heš-vrednost u heš-tabeli. One nastaju kada je broj potencijalnih ključeva dosta veći od dimenzije tabele.
Tada je bitno rešiti dva problema:

- odabir heš-funkcije koja će dovesti do što manje kolizija
- razrešavanje kolizija kada one nastanu.

Na broj kolizija utiče i veličina tabele pa se ona često proširuje kada broj kolizija postane preveliki. Međutim, čak i da zauzmemo praktično celu memoriju, kolizije nastaju već kad je niz gotovo prazan (za milion elemenata, već nakon oko 1000 elemenata (jednog promila) imamo veću verovatnoću da će se kolizije javiti nego da se neće javiti). Stoga je jako bitno da razrešavamo kolizije.

Jedan od načina da se kolizije razreše je **odvojeno ulančavanje**. Ono podrazumeva da se na svakom polju heš-tabele formira <u>povezana lista</u> koja sadrži sve vrednosti koje se slikaju na tu poziciju. Te vrednosti obavezno moraju da sadrže ključ, a mogu i da sadrže neke druge podatke (kod skupa je dovoljno samo ključ, a kod mape možemo čuvati i vrednost pridružene ključu). Sada u samom nizu (heš-tabeli) čuvamo pokazivače na elemente ovih listi, pri čemu nekada možemo uštedeti i vreme i memoriju ako umesto pokazivača na prvi element liste čuvamo sam prvi element (jer će se u velikom broju slučajeva kod dobrih heš-funkcija na nekoj poziciji nalaziti samo jedan ili nijedan element). Bitno je samo da znamo da razlikujemo popunjene i prazne pozicije.

<u>Pretraga elemenata</u> sa datim ključem se vrši tako što se odredimo heš-vrednost datog ključa i onda vršimo linearnu pretragu elemenata liste na toj poziciji u heš-tabeli. <u>Umetanje elemenata</u> se vrši na početak liste (ako duplikati nisu dozvoljeni, prvo se linearnom pretragom proverava da li element već postoji u listi i ako su u pitanju skupovi, konstatuje se da ništa ne treba da se uradi, a ako je u pitanju mapa, možemo ažurirati vrednost ključa). <u>Brisanje elemenata</u> se vrši tako što prvo linearnom pretragom nađemo traženi element u listi i onda ga obrišemo.

U realnim implementacijama, kada liste postanu predugačke, vrši se realokacija heš-tabele da bi se održala efikasnost rada sa heš-tabelama. Naravno, ako je heš-funkcija loša, ovo ne rešava problem i zato je važno da budu ravnomerne.

Složenost ovih operacija je linearna, međutim ako pretpostavimo da imamo ravnomernu funkiju i da liste neće biti dugačke, možemo smatrati da je složenost ovih operacija konstantna pri čemu konstantni faktor ne mora biti mali. Naravno, složenost najgoreg slučaja (kada su svi elementi u jednoj listi, neravnomerna funkcija) je ***O(n)***, ali je verovatnoća da će se to desiti izuzrtno mala.

18. Sortiranje uz pomoć hipa – formiranje hipa naniže

Ovaj algoritam je modifikacija sortiranja selekcijom. Imamo dve faze:

- *Prva faza* - formiramo maks-hip od svih elemenata niza koji sortiramo
- *Druga faza* - uklanjamo jedan po jedan najveći element i stavljamo ga na kraj rezultujućeg niza

Možemo koristiti samo 1 niz za čuvanje elemenata originalnog niza, formitanje hipa i čuvanje sortiranog niza.

Postoje dve varijante formiranja maks-hipa:

- **formiranje hipa naniže** (**Vilijamsov metod**)
- **formiranje hipa naviše** (**Flojdov metod**) - efikasniji ali na veoma velikim nizovima

Formiranje hipa naniže:

- Prva faza:
  - Krenemo od praznog hipa i ubacujemo jedan po jedan element niza u hip koji raste malo po malo
  - Invarijanta: na prvih *k* mesta u nizu se nalazi formiran hip od prvih *k* elemenata polaznog niza, dok se na ostalim mestima nalaze nepormenjeni elementi polaznog niza
  - Složenost: $\Theta(nlog\ n)$
- Druga faza:
  - Nakon što je hip formiran, uklanja se jedan po jedan najveći element i prebacuje se na kraj niza
  - Invarijanta: Nakon *k* koraka početnih *k* elemenata predstavljaju ispravan hip sastavljen od *k* najmanjih elemenata početnog niza, a na preostalih *n-k* pozicija se nalaze preostali elementi u sortiranom redosledu
  - Složenost: $\Theta(nlog\ n)$

---

19. Induktivno-rekurzivna konstrukcija – primeri: Grejovi kodovi, treći obilazak binarnog drveta

Primer: Grejovi kodovi - Grejov kôd reda *n* podrazumeva ređanje svih *n*-tocifrenih binarnih zapisa tako da se svaka dva susedna zapisa razlikuju tačno u jednom bitu. Grejov kod dužine 0 je prazna niska. Potrebno je odrediti *k*-ti po redu Grejov kod dužine *n*

Definišemo funkciju koja određuje *k*-ti po redu zapis Grejovog koda dužine *n*, koji ima 2^n brojeva (podrazumevaćemo da je $0 \leq k < 2^n$). Ako je *n* nula, rezultat je prazna niska. Pošto ukupno

ima 2^n kodova, elementi u prvoj polovini su na pozicijama $0 \leq k < 2^{n-1}$, dok su elementi u drugoj polovini na pozicijama $2^{n-1} \leq k < 2^n$.

- Kada je $0 \leq k < 2^{n-1}$, tada se vraća *k*-ti element Grejovog koda dužine *n-1* dopunjen početnom nulom
- Kada je $2^{n-1} \leq k < 2^n$, tada se vraća $(2^n - 1 - k)$-ti element Grejovog koda dužine *n-1* dopunjen početnom jedinicom (pozicija *k* se svodi u raspon $[0,\ 2^{n-1})$ operacijom $k - 2^{n-1}$ i zatim moramo i da obrnemo redosled elemenata. Prilikom obrtanja redosleda elemenata, svaka pozicija p u intervalu [0,m) se preslikava u poziciju m − p − 1 pa se pozicija $k - 2^{n-1}$ preslikava u poziciju $2^{n-1} - (k - 2^{n-1}) - 1 = 2^n - 1 - k$).

Ovo možemo uraditi i iterativno tako što krenemo od praznog stringa i ažuriramo vrednosti *n* i *k* na isti način kao i pre.

---

20. Induktivno-rekurzivna konstrukcija, ojačavanje IH – primer: maksimalni zbir nesusednih elemenata niza

Često tokom induktivno-rekurzivne konstrukcije ima smisla uvesti još neku veličinu koja se ne traži na samom kraju, ali nam pomaže da se napravi induktivni korak i tu tehniku zovemo **ojačavanje iduktivne hipoteze**. Kada uvedemo tu dodatnu vrednost, moramo da proširimo i bazu indukcije i da umemo da izračunamo i bazu za tu vrednost i narednu vrednost.

$A_0 \wedge (\forall n)(A_n \Rightarrow A_{n+1}) \Rightarrow (\forall n)A_n$
$(A_0 \wedge B_0) \wedge (\forall n)((A_n \wedge B_n) \Rightarrow (A_{n+1} \wedge B_{n+1})) \Rightarrow (\forall n)A_n$

Primer: Maksimalni zbir nesusednih elemenata niza

<u>Induktivno-rekurzivna konstrukcija</u>:

- Maksimalni zbir nesusednih elemenata praznog niza je 0
- Ako je niz neprazan, razlažemo ga na prefiks i poslednji element
    - rekurzivno možemo da izračunamo maksimalni zbir nesusednih elemenata prefiksa, ali nam ova informacija ne govori ništa o tome da li poslednji element sme ili ne sme biti uključen u zbir pa moramo ojačati induktivnu hipotezu

<u>Ojačavanje induktivne hipoteze</u>:

- Ako je poslednji element prefiksa uključen u maksimalni zbir nesusednih elemenata prefiksa, onda poslednji element niza ne sme biti uključen (jer su poslednji element niza i prefiksa susedni elementi)

- Potrebno je tada da znamo i koji je maksimalni zbir nesusednih elemenata prefiksa kada poslednji element prefiksa nije uključen u taj zbir

<u>Pojacana induktivna hipoteza</u>:

- Za prefiks niza umemo da izračunamo:
  - maksimalni zbir nesusednih elemenata (M)
  - maksimalni zbir nesusednih elemenata bez poslednjeg elementa (M^{bez})
- Baza indukcije (prazan niz):
  - $M = 0$
  - $M^{bez} = 0$
- Induktivni korak:
  - Na osnovu podataka o prefiksu (M, M^{bez}) računamo podatke za ceo niz (M', M'^{bez})
    - Ako je poslednji element uključen, maksimalni zbir koji možemo da dobijemo je $M^{bez} + a_{n-1}$
    - Ako poslednji element nije uključen, maksimalni zbir koji možemo da dobijemo je M
  - $M' = max(M,\ M^{bez} + a_{n-1})$
  - $M'^{bez} = M$

a_i	M	M^{bez}
	0	0
7	7	0
3	7	7
2	9	7
4	11	9
1	11	11
5	16	11

21. Podeli-pa-vladaj – primer: brzo parcijalno sortiranje (QuickSelect)

QuickSelect

QuickSelect je modifikacija algoritma QuickSort koji se koristi za razne srodne probleme (ispis *k* najvećih/najmanjih elemenata, zbir *k* najvećih/najmanjih, određivanje *k*-ti po veličini element niza,...). Koristimo ovaj algoritam jer ako je *k* malo a *n* veliko, ne isplati nam se da sortiramo ceo niz da bismo pronašli *k* najmanjih/najvećih (npr. ako imamo 1000000 el. a traži nam se 10 najvećih).

Primer: Zbir *k* najvećih elemenata niza

Mogli bismo da sortiramo niz, ali to je suvišno. Dovoljno je da *k* najvećih elemenata dovedemo na početak niza (u proizvoljnom redosledu), iza kojih slede ostali elementi niza (isto u proizvoljnom redosledu).

Ako je *k=0,* vraćamo 0, ako je *k=l-d+1* tada vraćamo zbir svih elemenata iz intervala [l, d]. Pivotiramo niz:

- Ako je broj elemenata levo od pivota *p-l* veći ili jednak broju *k*, zanemarujemo elemente desno od pivota uključujući pivot i vraćamo zbir *k* najvećih u delu niza [l, p-1].
- Ako je broj elemenata levo od pivota *p-l* strogo manji od *k*, vraćamo zbir svih elemenata levo od pivota uključujući pivot tj. zbir elemenata iz intervala [l, p], i tražimo preostalih *k-(p - l + 1)* elemenata desno od pivota tj. u intervalu [p+1, d]

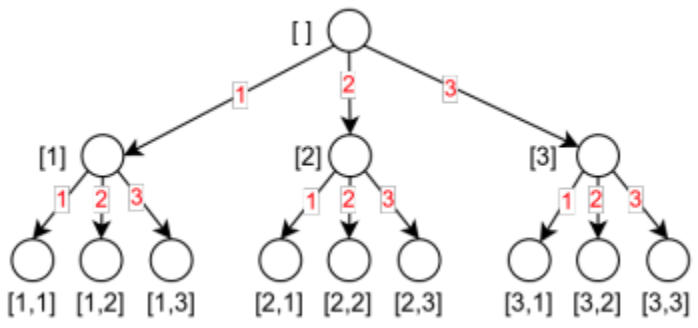
U obe grane rekurzije vršimo samo 1 rekurzivni poziv (za razliku od QuickSort-a koji vrši 2).
<u>Složenost</u>: Ako pretpostavimo da pivot deli niz na dva dela približno jednake dimenzije, složenost ovog algoritma možemo predstaviti jednačinom $T(n) = T(\frac{n}{2}) + O(n)$, $T(0) = O(1)$ čije je rešenje ***O(n)***.

---

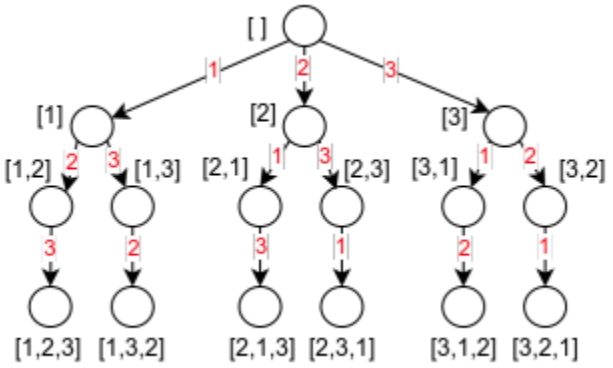
22. Generisanje kombinatornih objekata: rekurzivno nabrajanje – primeri (varijacije, podskupovi)

Primer: Sve varijacije - generisati sve varijacije dužine *k* skupa {1,...,n}

Npr. *k=2,*
skup {1, 2, 3}

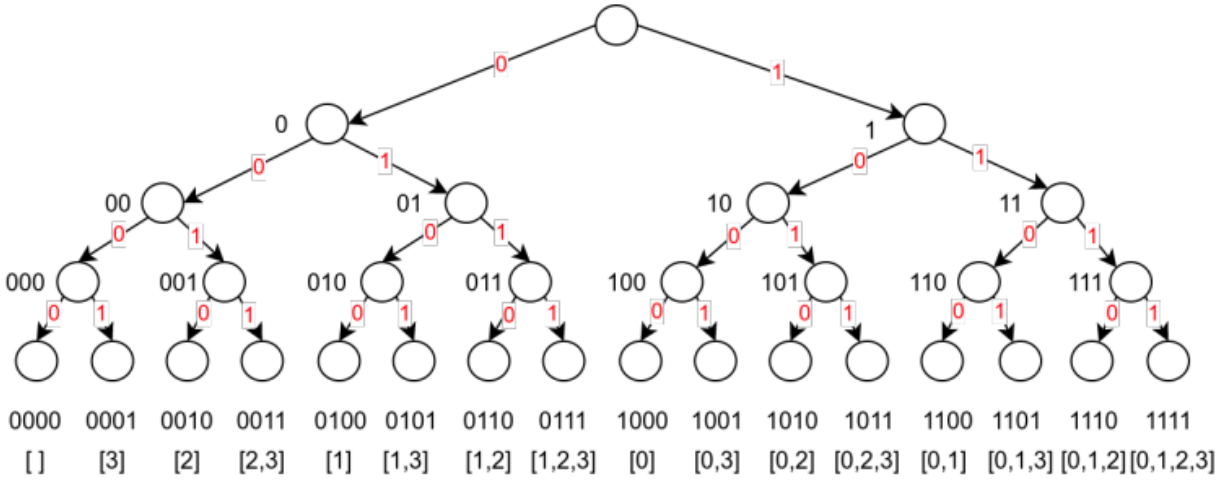


Varijacije bez ponavljanja:

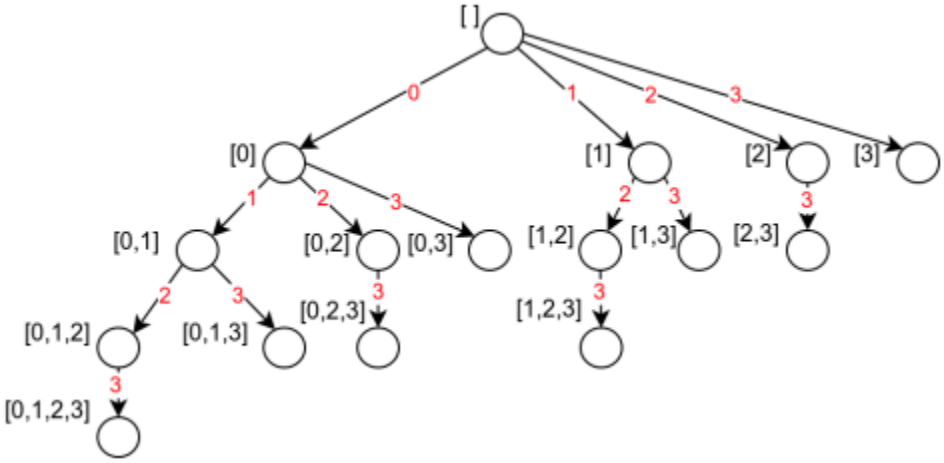


Primer: Svi podskupovi - generisati sve podskupove datog skupa

Generisanje svih podskupova je vezano za generisanje svih varijacija, ako su podskupovi predstavljeni nizom 0 i 1. Sada treba generisati sve varijacije dužine *n* skupa {0,1}



Sve podskupove skupa {0,...,n-1} u leksikografskom poretku možemo generisati rekurzivnom funkcijom koja prima tekući podskup, obrađuje ga i pokušava na sve moguće načine da ga proširi. Pošto skupove predstavljamo pomoću strogo rastuće sortiranog niza, tekući podskup proširujemo redom svim elementima koji su strogo veći od njegovog poslednjeg elementa i manji od *n*. Krećemo od praznog niza i proširujemo ga svim elementima iz početnog skupa redom.



23. Generisanje kombinatornih objekata: rekurzivno nabrajanje – primeri (kombinacije, particije)

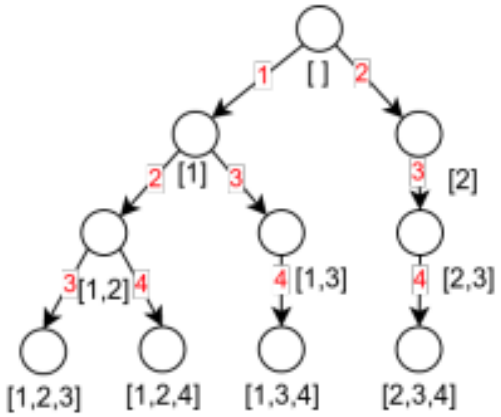
Primer: Kombinacije bez ponavljanja

Kombinacije bez ponavljanja se javljaju prilikom izvlačenja loto kuglica. Imamo dva moguća rekurzivna načina za generisanje:

- Generisanje po pozicijama (za svaku poziciju u rezultujućoj kombinaciji određujemo moguće vrednosti brojeva)
- Generisanje po vrednosti (za svaku vrednost određujemo da li jeste ili nije u kombinaciji)

Generisanje po pozicijama

Idemo redom po pozicijama kombinacije i popunjavamo ih. Npr. ako biramo 3 od 4 elemenata:



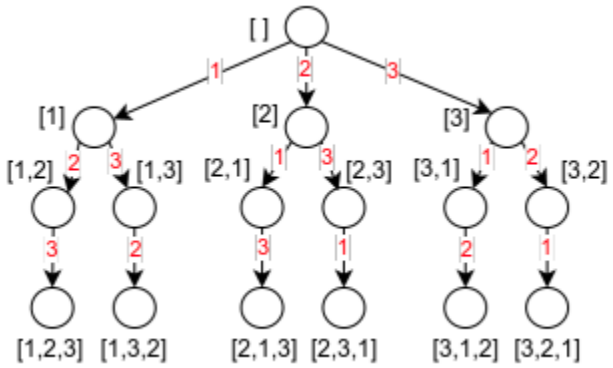
Na poziciju i niza dužine *k* se mogu staviti svi elementi iz intervala [poc, kraj]

- $poc = a_i + 1,\ i > 0$ tj. $poc = 1,\ i = 0.$
- najveći broj koji može biti na poziciji $i = k - 1$ je $kraj = n$, za $i = k - 2$ je $kraj = n - 1$ itd. Vidimo da važi da je $kraj - i = n - (k - 1)$, pa je najveći broj koji može biti na poziciji i jednak $kraj = n + i - k + 1.$

24. Generisanje kombinatornih objekata: rekurzivno nabrajanje – primeri (permutacije)

Postupak generisanja permutacija je isti kao postupak generisanja varijacija bez ponavljanja. Obrađujemo jednu po jednu poziciju i na nju stavljamo one elemente skupa {1,...,n} koji već nisu

iskorišćeni. Da bismo izbegli linearnu pretragu, funkciji prosleđujemo i vektor boolova u kome se za svaki element označava da li je već iskorišćen ili nije.



---

25. Pretraga sa povratkom – primer: n-dama

Potrebno je postaviti n dama na šahovsku tablu tako da se nikoje dve međusobno ne napadaju.

Jedno rešenje bi bilo pretraga grubom silom. Mogli bismo da ispitujemo sve moguće rasporede dama ali primetimo da odmah možemo da nametnemo uslove koji će nam znatno smanjiti pretragu. Ako stavimo uslov da u svakoj vrsti i svakoj koloni može biti najviše jedna dama (biramo n različitih brojeva od 1 do n), sada ispitujemo sve permutacije kojih ima n!. Sada je dovoljno samo ispitati uslov da li se dve dame napadaju po dijagonalama.

Razlika izmedju grube sile i bektrekinga je što će se kod grube sile zaista izgenerisati sve moguće permutacije pa tek onda proveravati njihova korektnost, dok ćemo kod bektrekinga dok generišemo vršiti proveru korektnosti i ako ustanovima da parcijalno popunjena permutacija ne može dati ispravan raspored, vršimo odsecanje u pretrazi. Na primer, u mnogim slučajevima će se veoma rano primetiti da su dame na istoj dijagonali i cela ta grana pretrage biće odsečena.

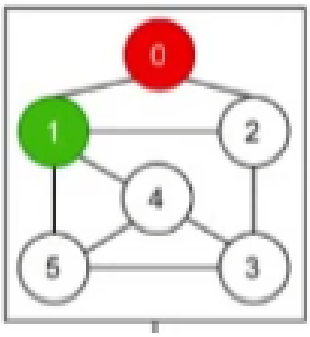
---

26. Pretraga sa povratkom – primer: bojenje grafa sa 3 boje

Treba proveriti da li je moguće obojiti čvorove nekog grafa pomoću 3 boje tako da su susedni čvorovi obojeni različitim bojama. Grubom silom bismo određivali sve moguće mogućnosti bojenja, pa tek na kraju proveravali da li postoje dva čvora koja su obojena istom bojom. Ovo nije pametno jer mnogo ranije možemo odrediti da trenutno bojenje nije validno, ako prilikom dodeljivanja boje novom čvoru vidimo da se desilo da susedni čvorovi imaju istu boju, odmah vršimo odsecanje (koristimo bektreking).

Ako primetimo da je problem simetričan (ako je neko bojenje moguće, možemo zameniti imena boja i bojenje će i dalje biti moguće), onda možemo pretpostaviti da je prvi čvor obojen fiksiranom bojom 1. Takođe, njegovom prvom susedu takođe možemo fiksirati početnu boji na npr. boju 2. Nastavljamo ispitivanje tek od trećeg čvora i pokušavamo razne boje.

Primer:



---

27. Dinamičko programiranje: pojam, pristupi – primer: Fibonačijev niz

U praksi prilikom izvršavanja rekurzivne funkcije često dolazi do **preklapanja reukrzivnih poziva**, tj. identični rekurzivni pozivi se izvršavaju više puta. Ako se to dešava često program postaje neefikasan (često eksponencijalne složenosti). Tehnika **dinamičkog programiranja** pruža efikasnije rešenje problema tako što vremensku složenost popravlja angažovanjem dodatne memorije u kojoj se beleže rezultati izvršenih rekurzivnih poziva. Dinamičko programiranje dolazi u dva oblika:

- **Memoizacija** ili **dinamičko programiranje naniže** zadržava rekurzivnu definiciju ali u dodatnoj strukturi podataka (najčešće u nizu ili matrici) beleži rezultate rekurzivnih poziva da bi u narednim pozivima u kojima su parametri isti samo očitala iz te strukture.
- **Dinamičko programiranje naviše** u potpunosti uklanja rekurziju (koja se zamenjuje induktivnom konstrukcijom) i pomoćnu strukturu podataka popunjava iscprno u nekom sistematičnom redosledu.

Kod dinamičkog programiranja naviše jasno je da se izračunavaju vrednosti funkcije za sve moguće parametre manje od traženog, dok se kod memoizacije može desiti da se funkcija ne poziva za neke parametre. Iz tog razloga možemo pomisliti da je memoizacija efikasnija ali u praksi je uglavnom potrebno računati vrednost rekurzivne funkcije za veoma veliki broj različitih parametara pa se prednost memoizacije retko sreće.

Primer: Fibonačijev niz

- Klasična rekurzivna implementacija već pri računanju 50. člana Fibonačijevog niza radi veoma sporo jer se funkcija za iste parametre poziva veliki broj puta. Složenost je eksponencijalna.

```
long long fibonaci(int n) {
      if (n == 0 || n == 1)
            return 1;
      return fibonaci(n-1) + fibonaci(n-2)
}
```

- Tehnikom dinamičkog programiranja naniže (memoizacijom) kroz globalni niz čuvamo vrednosti već izračunatih rekurzivnih poziva i na početku svakog poziva proveravamo da li tu vrednost imamo već sačuvanu.
  Potrebno je nekako označiti vrednosti parametara u nizu za koje još ne znamo rezultate rekurzivnih poziva. U slučaju da unapred znamo da su npr. sva rešenja nenegativna, možemo koristiti -1 ili ako su pozitivna 0. Ako nemamo takvih pretpostavki, možemo napraviti dodatni niz logičkih vrednosti kojima čemo ekspicitno kodirati da li za neki parametar znamo ili ne znamo vrednost.
  Za niz *memo* od *n* elemenata treba alocirati *n+1* element. Složenost sada više nije eksponencijalna, već linearna.

```
vector<long long> memo(n+1, -1);

long long fibonaci(int n) {
      if (memo[n] != -1)
            return memo[n];
      if (n == 0 || n == 1)
            return memo[n] = 1;
      return memo[n] = fibonaci(n-1) + fibonaci(n-2)
}
```

- Tehnikom dinamičkog programiranja naviše potpuno eliminišemo rekurziju i sami popunjavamo pomoćnu strukturu od početka do kraja za sve potrebne vrednosti. Krajnja vrednost će na kraju biti na poziciji *n* pomoćne strukture.

```
long long fibonaci(int n) {
      vecotr<long long> dp(n+1);
      dp[0] = dp[1] = 1;
      for (int i = 2; i <= n; i++)
            dp[i] = dp[i-1] + dp[i-2];
```

```
	return dp[n];
}
```

- Kod dinamičkog programiranja naviše možemo izvršiti dodatnu memorijsku optimizaciju jer primećujemo da ne moramo pamtiti sve vrednosti, već samo dva prethodnika.

```
long long fibonaci(int n) {
	long long prethodni = 1, pretprethodni = 1;
	for (int i = 2; i <= n; i++) {
		long long tekuci = prethodni + pretprethodni;
		pretprethodni = prethodni;
		predhodni = tekuci;
}
	return prethodni;
}
```

---

28. Dinamičko programiranje: optimizacioni problemi – primer: edit-rastojanje

Dinamičko programiranje se često primenjuje u rešavanju optimizacionih zadataka (zadataka u kojima se traži najmanja ili najveća vrednost koja zadovoljava neki uslov). Da bi optimizacioni problem mogao da se rešava dinamičkim programiranjem, on mora da zadovoljava svojstvo **optimalne podstrukture** (optimalno rešenje polaznog problema se može odrediti preko optimalnih rešenja potproblema istog oblika ali manje dimenzije).

Na primer, najkraći put od tačke A do tačke B prolazeći kroz tačku C se može odrediti kombinovanjem najkraćeg puta između tačke A tačke C i najkraćeg puta između tačke B i tačke C.
Međutim, najjeftiniji let od aerodroma A do aerodroma B preko aerodroma C ne mora biti kombinacija najjeftinijeg leta od tačke A do tačke C i najjeftinijeg leta od tačke B do tačke C (zbog posebnih popusta koje avio kompanije nude za povezane letove).

Primer: Edit rastojanje - imamo 2 niske i koliko nam je operacija potrebno od jedne da dobijemo drugu ako su na dozvoljene operacije umetanja slova, brisanja slova i izmena slova.

<u>Rekurzivno rešenje</u>:
- Ako je prva niska prazna moramo da joj dodamo sve karaktere druge niske
- Ako je druga niska prazna moramo da joj dodamo sve karaktere prve

- Ako su obe niske neprazne, prva je oblika xa a druga yb za neke niske x i y i neke karaktere a i b.
  - ako su karakteri a i b jednaki, rekurzivno pretvaramo nisku x u nisku y
  - u suprotnom je potrebno nekako postići da se reči završavaju istim karakterom
    - možemo obrisati karakter a iz niske xa i zatim rekurzivno pretvoriti nisku x u nisku yb
    - možemo dodati karakter b na kraj prve niske i zatim rekurzivno pretvoriti nisku xa u nisku y
    - možemo izmeniti karakter a u karakter b i zatim rekurzivno pretvoriti nisku x u nisku y

    Biramo najbolju od prethodne tri opcije.

Problem sa ovim pristupom je što dolazi do velikog broja ponovljenih rekurzivnih poziva i složenost je eksponencijalna. Ovo rešavamo **dinamičkim programiranjem** (ili memoizacijom ili dp naviše). I jedan i drugi pristup zahtevaju formiranje matrice dimenzija $(n_1 + 1) \cdot (n_2 + 1)$ gde su n_1 i n_2 redom dimenzije prve i druge niske i zatim upisujemo rezultate rekurzivnih poziva u tu matricu (memoizacija) odnosno popunjavamo matricu redom do krajnjeg dole desnog elementa (dp naviše). Složenost je sada ***O(***$n_1 \cdot n_2$***)***.

Kod memoizacije, prilikom ulaska u rekurzivni poziv prvo proveravamo da li već imamo izračunatu vrednost za taj poziv i ako imamo samo je čitamo iz matrice, a inače pri svakom vraćanu iz rekurzivnog poziva usput i upisujemo vrednost na odgovarajuće mesto u matrici.
Kod dp naviše, popunjavamo iterativno matricu i prva vrsta i kolona odgovaraju izlazu iz rekurzije (zato imamo za jedan više vrsta i kolona), a pošto svaki element zavisi od elementa gore-levo, elementa gore i elementa levo, možemo popunjavati vrsta po vrstu ili kolona po kolonu.

---

29. Dinamičko programiranje: prebrojavanje kombinatornih objekata – primeri (kombinacije, particije,...)

Dinamičko programiranje možemo koristiti prilikom raznih problema prebrojavanja.

Primer: Broj kombinacije
Možemo iskoristiti algoritam generisanja kombinacija po vrednosti, odnosno za svaku vrednost određujemo da li jeste ili nije uključena u kombinaciju, međutim primećujemo da ima dosta ponovljenih rekurzivnih poziva što dovodi do eksponencijalne složenosti.
Implementaciju najlakše možemo popraviti primenom tehnike **memoizacije**. Pošto imamo dva parametra koja se šalju kroz rekurzivnu funciju, pravimo matricu i popunjavamo je. Prilikom ulaska u rekurzivni poziv prvo proveravamo da li imamo već vrednost izračunatu u matrici i ako imamo samo je vraćamo, a ako nemamo prilikom izračunavanja i vraćanja vrednosti upisujemo je na odgovarajuće mesto u matrici. Ovo dovodi do složenosti ***O(nk)***. Možemo koristiti i tehniku **dp naviše** i iterativno popunjavati matricu. Ova tehnika se može primeniti na brojanje bilo kog kombinatornog objekta koje nabrajamo rekurzivnom funkcijom.